

ECOALERT - LORA BASED SMART WILDLIFE DETECTION AND ALERT SYSTEM

A PROJECT REPORT

*Submitted to the APJ Abdul Kalam Technological University
in partial fulfillment of requirements for the award of Degree of*

Bachelor of Technology
in
Electronics and Communication Engineering
by

NANDHANA K S (Reg. No. VAS21EC080)

GRACE BIJU (Reg. No. VAS21EC066)

INDRAJITH V L (Reg. No. LVAS21EC113)

SREEHARI M (Reg. No. VAS21EC097)



Department of Electronics and Communication Engineering
Vidya Academy of Science and Technology
Thrissur, Kerala

March 2025

DECLARATION

We hereby declare that the project report **ECOALERT - LORA BASED SMART WILDLIFE DETECTION AND ALERT SYSTEM**, submitted for partial fulfillment of the requirements for the award of degree of Bachelor of Technology of the APJ Abdul Kalam Technological University, Kerala is a bonafide work done by us under supervision of **Anju Vincent** and **Dr. Abitha M A** and **Anil M**

This submission represents our ideas in our own words and where ideas or words of others have been included, we have adequately and accurately cited and referenced the original sources.

We also declare that I have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in my submission. We understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma or similar title of any other University.

Thrissur
13-03-2025

GRACE BIJU
SREEHARI M
INDRAJITH V L
NANDHANA K S

**DEPT. OF ELECTRONICS & COMMUNICATION ENGINEERING
VIDYA ACADEMY OF SCIENCE AND TECHNOLOGY, THRISSUR**

2024-25



CERTIFICATE

This is to certify that the report entitled **ECOALERT - LORA BASED SMART WILDLIFE DETECTION AND ALERT SYSTEM** submitted by **NANDHANA K S** (VAS21EC080), **INDRAJITH V L** (LVAS21EC113), **GRACE BIJU** (VAS21EC066) & **SREEHARI M** (VAS21EC097) to the APJ Abdul Kalam Technological University in partial fulfillment of the B.Tech. degree in Electronics & Communication Engineering is a bonafide record of the project work carried out by him under our guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Anju Vincent
(Project Guide)
Assistant Professor
Dept.of ECE
Vidya Academy of Science and
Technology, Thrissur

Anil M
(Project Co-Ordinator)
Assistant Professor
Dept.of ECE
Vidya Academy of Science and
Technology, Thrissur

Dr. S. Swapna Kumar
Professor and Head
Dept.of ECE
Vidya Academy of Science and Technology
Thrissur

Abstract

Animal-train collisions present a significant challenge in regions where animals cross railway tracks. Such accidents result in loss of life and damage to railway infrastructure. The proposed system employs a MobileNet V2 CNN-based early warning system to mitigate these incidents. The system utilizes CNN-based image processing, along with an IR sensor and a camera, to detect animals. Multiple detection units are deployed along railway tracks. The prototype system processes data using a laptop as a substitute for a Raspberry Pi, with a NodeMCU controlling components such as GSM, GPS, and LoRa modules. An alert is transmitted to the train, where an LCD display and a buzzer notify the locomotive pilot to take appropriate action. The system offers an efficient solution to prevent collisions, thereby enhancing railway safety and reducing fatalities.

Acknowledgement

We take this opportunity to express my deepest sense of gratitude and sincere thanks to everyone who helped us to complete this work successfully. We express our sincere thanks to Dr. S. Swapna Kumar, Head of Department, Electronics and Communication Engineering, Vidya Academy of Science and Technology, Thrissur for providing us with all the necessary facilities and support.

We would like to express my sincere gratitude to Anil M, department of Electronics and Communication Engineering, Vidya Academy of Science and Technology Thrissur for the support and co-operation.

We would like to place on record my sincere gratitude to our project guide Anju Vincent, Assistant Professor, Electronics and Communication Engineering, Vidya Academy of Science and Technology for the guidance and mentorship throughout this work.

Finally I thank my family, and friends who contributed to the successful fulfilment of this seminar work.

GRACE BIJU
SREEHARI M
INDRAJITH V L
NANDHANA K S

Contents

Abstract	ii
Acknowledgement	iii
List of Figures	vi
List of Tables	viii
List of Symbols and Abbreviations	ix
1 INTRODUCTION	1
1.1 PREAMBLE	1
1.2 PROJECT BACKGROUND AND MOTIVATION	1
1.3 SUMMARY	4
2 LITERATURE REVIEW	6
3 DESIGN METHODOLOGY	14
3.1 PREAMBLE	14
3.2 PROPOSED SYSTEM DESIGN	14
3.3 BLOCK DIAGRAM	15
3.4 FLOW CHART	18
3.5 CIRCUIT DIAGRAM	20
3.6 SUMMARY	22
4 SYSTEM DEVELOPMENT	23
4.1 HARDWARE	24

4.1.1	NODEMCU PROCESSOR	24
4.1.2	ARDUINO NANO PROCESSOR	30
4.1.3	LORA RA-02	35
4.1.4	JHD162A: 2x16 ALPHANUMERIC LCD MODULE	37
4.1.5	IR MODULE	41
4.1.6	NEO-6 GPS	44
4.1.7	GSM Module	46
4.2	SOFTWARE	51
4.2.1	ANACONDA NAVIGATOR	51
4.2.2	SPYDER IDE	53
4.2.3	ARDUINO IDE	54
4.2.4	MOBILENETV2	56
5	RESULTS AND DISCUSSION	61
5.1	PREAMBLE	61
5.2	RESULT	61
5.3	SUMMARY	70
6	CONCLUSION	71
6.1	PREAMBLE	71
6.2	CONCLUSION	71
6.3	ADVANTAGES	73
6.4	DISADVANTAGES	74
6.5	FUTURE SCOPE	75
6.6	SUMMARY	77
	REFERENCES	77

List of Figures

1.1	Estimated elephant population in India using various population estimation approaches.	3
1.2	Deaths of elephants & other wild animals on railway tracks in India	4
3.1	Block diagram of the transmitter	16
3.2	Block diagram of the receiver	17
3.3	Flow chart of the system	19
3.4	Circuit diagram of transmitter	20
3.5	Circuit diagram of receiver	21
4.1	NodeMCU processor	25
4.2	NodeMCU pinout	26
4.3	Function blockdiagram of NodeMCU	28
4.4	Arduino NANO processor	32
4.5	Arduino NANO pinout	33
4.6	LoRa module	36
4.7	LCD display	38
4.8	2-line by 16-character display	39
4.9	IR sensor	42
4.10	IR sensor pinout	43
4.11	Neo-6 GPS	44
4.12	GSM module	47
4.13	GSM architecture	49
4.14	Anaconda navigator	52
4.15	Spyder IDE	54

4.16	Arduino IDE	56
4.17	MobileNetV2 based deep learning model for image classification . . .	57
5.1	System	62
5.2	IR triggered and starting image processing	64
5.3	CNN detecting an elephant and a person together	65
5.4	CNN detecting a cow	66
5.5	Alert received through SMS with location of the detected place	67
5.6	LCD display showing alert message	68
5.7	CNN detection accuracy for selected classes	69

List of Tables

4.1	NodeMCU Pinout	27
4.2	Technical specifications of NodeMCU module	30
4.3	LoRa pinout	37
4.4	GSM details	48

List of Symbols and Abbreviations

CNN	Convolutional Neural Network
IR	Infrared Radiation
UART	Universal Asynchronous Receiver/Transmitter
TX	Transmitter Data
RX	Receiver Data
PV	Photovoltaic
IDE	Integrated Development Environment
GPS	Global Positioning System
GSM	Global System for Mobile Communication
GPRS	General Packet Radio Service

Chapter 1

INTRODUCTION

1.1 PREAMBLE

The Human-Elephant Conflict become a pressing issue, particularly where elephant habitats overlap with human infrastructure, leading to tragic elephant-train collisions. These incidents often result in the death or severe injury of endangered Asian elephants, creating orphaned or disabled animals, while also causing significant financial losses and service disruptions for railway operations. Key factors contributing to these collisions include limited reaction time due to poor visibility at night, sharp track turns, and adverse weather conditions. Investigations show that most incidents occur at specific “hotspots” where elephant pathways intersect railway tracks. Leveraging advancements in Convolutional Neural Network (CNN) technology, this project proposes an image-based detection system trained on a curated dataset of elephants. This vision-based early warning prototype aims to detect elephants in real-time and alert train operators, with the potential to scale into a network that could prevent collisions, protect elephant populations, reduce economic losses, and enhance both wildlife conservation and railway safety.

1.2 PROJECT BACKGROUND AND MOTIVATION

The Human-Elephant Conflict is a growing issue driven by the expansion of human communities and farmlands into areas where elephants have roamed for centuries. As

these natural migration paths are increasingly intersected by railways and other human-made structures, encounters between elephants and trains have become more common and often tragic. When an elephant collides with a train, the impact is devastating: elephants are seriously injured, killed, or separated from their herds, and railway services suffer from delays, damages, and costly disruptions. While efforts like electric fences and habitat relocations have helped to a degree, these solutions aren't enough when it comes to preventing collisions on railway tracks. Such incidents happen most often in specific, high risk areas, where elephants habitually cross railway lines on their routes to feeding grounds or water sources. Traditional methods can't fully address this unique risk, leaving a crucial need for an approach that specifically targets elephant safety on railway tracks.

The motivation behind this project comes from the potential to use advanced technology to protect both wildlife and human infrastructure. By applying a convolutional neural network (CNN) in a way that allows real-time detection, this system aims to create an early warning signal for train drivers approaching elephant crossing zones. Using strategically placed cameras both visible light and infrared—the system is designed to detect elephants under all kinds of lighting and weather conditions. When an elephant is detected, an alert can immediately prompt the train driver to slow down or stop, potentially preventing a collision altogether. This project is ultimately about finding a balance where human progress can coexist with wildlife conservation. By reducing elephant fatalities and enhancing railway safety, the project strives to create a practical, compassionate solution to a challenge where both human and animal lives are at stake.

In Figure 1.1, the graph depicting trends in elephant and human populations from 1980 to 2020 illustrates key insights into the growing Human-Elephant Conflict (HEC). The elephant population, represented by the blue line, increased until the mid-1990s and then stabilized at around 30,000. This stabilization may reflect the challenges elephants face, such as habitat loss, poaching, and limited resources, despite conservation efforts. Elephants require vast, uninterrupted habitats, and when these areas are fragmented, their numbers can plateau or decline.

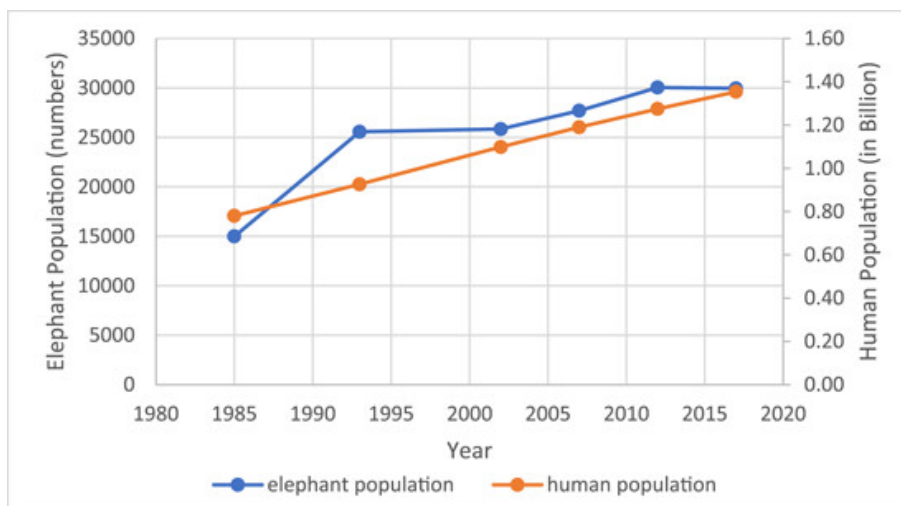


Figure 1.1: Estimated elephant population in India using various population estimation approaches.

In contrast, the human population, represented by the orange line, shows a continuous, steep increase from below 1 billion in 1980 to about 1.4 billion by 2020. This growth can be linked to improved healthcare, technology, and living conditions, driving the expansion of human settlements, infrastructure, and agriculture into natural habitats. This expansion creates direct competition for space and resources between humans and wildlife, exacerbating HEC.

The mid-1990s mark a critical period where human population growth overtakes that of the elephant population, symbolizing increased pressure on natural ecosystems. As human activities encroach on elephant corridors, conflicts become more frequent, resulting in crop damage, property destruction, and even fatalities on both sides. These trends highlight the need for strategies that balance human development with wildlife conservation, such as the creation of wildlife corridors and early warning systems. By addressing habitat fragmentation and promoting coexistence, these strategies aim to reduce conflict and support sustainable conservation efforts.

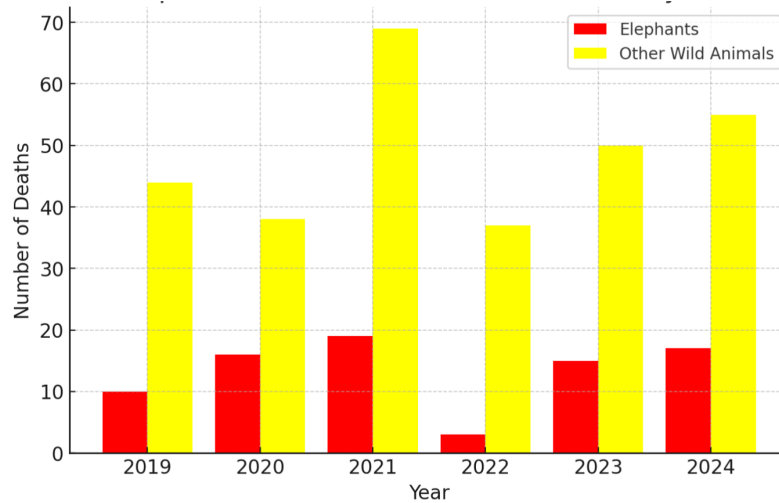


Figure 1.2: Deaths of elephants & other wild animals on railway tracks in India

Figure 1.2 illustrates the number of deaths of elephants and other wild animals on railway tracks in India from 2019 to 2024.

- General Trend:

The number of deaths fluctuates over the years for both elephants and other wild animals. Wild animals consistently have a higher number of deaths compared to elephants.

- Elephant Deaths:

The deaths of elephants remain relatively lower than other wild animals. The highest number of elephant deaths is observed in 2021, followed by a decline in 2022. The deaths increased again in 2023 and 2024 but remain lower than in 2021.

- Deaths of Other Wild Animals:

The numbers are significantly higher than those of elephants. A sharp peak is observed in 2021, where the deaths of wild animals reach nearly 70. The lowest count for wild animal deaths is in 2022, but it rises again in 2023 and 2024.

1.3 SUMMARY

The Elephant Train Collision Prevention System is a comprehensive initiative aimed at reducing accidents between elephants and trains, blending advanced technology

with community engagement to ensure both wildlife conservation and rail safety. The system utilizes thermal cameras and motion sensors to detect elephants near railway tracks, even in low-light conditions, while GPS tracking monitors elephants equipped with collars, providing real-time movement data. A centralized platform collects this information and alerts train operators, allowing them to slow down or stop trains when elephants are detected. To deter elephants from crossing tracks, warning lights and auditory signals are deployed, redirecting them to safer routes. Additionally, safe crossing points like overpasses and underpasses are constructed in key migration corridors, providing elephants with secure alternatives that mimic natural environments. Community awareness programs complement these measures by educating local populations about elephant behavior, rail safety, and the importance of conservation, fostering collaboration between wildlife authorities, railway operators, and residents. By integrating technology, infrastructure, and community participation, the system effectively reduces train-elephant collisions while protecting migration routes and promoting coexistence between humans and wildlife.

Chapter 2

LITERATURE REVIEW

[1] Ranjith et al. (2022), in “An IoT-Based Monitoring System to Detect Animals on Railway Tracks Using Deep Learning Neural Networks,” propose an AI-driven system that utilizes deep learning neural networks and IoT sensors to detect animals on railway tracks in real time. The system integrates multiple sensor technologies, including cameras, ultrasonic, thermal, and infrared sensors, to ensure accurate monitoring of animal movements. By leveraging cloud-based AI processing, the system analyzes incoming data and immediately sends alerts to train operators to prevent potential collisions. The study highlights how AI-powered automation enhances railway safety by providing early warnings and reducing train-animal accidents, which are particularly common in wildlife-prone areas. Wireless communication networks, including LoRa and GSM, play a crucial role in delivering real-time alerts, making the system highly effective even in remote regions. The researchers emphasize that their approach significantly improves train safety, reduces financial losses from railway disruptions, and helps with wildlife conservation. Additionally, the study explores how integrating machine learning algorithms improves object classification and detection accuracy under various environmental conditions such as fog and low visibility. Future work could incorporate drone surveillance and edge computing for even faster processing and improved detection efficiency. Their research offers a scalable, cost-effective solution for railway safety.

[2] Sabnis & Lokeshkumar (2019), in “A Novel Object Detection System for Improving Safety at Unmanned Railway Crossings,” present an AI-based system

designed to enhance safety at unmanned railway crossings by detecting objects such as vehicles, animals, and pedestrians. The system employs computer vision techniques combined with IoT-based sensors to analyze real-time video feeds and identify potential hazards near railway tracks. The authors utilize deep learning algorithms to process images and classify objects accurately, triggering automated alerts to train operators and railway authorities. Their study emphasizes the importance of this system in remote areas where human supervision is minimal, making AI-powered monitoring essential for accident prevention. The model is optimized for different environmental conditions, ensuring reliable detection even in poor lighting or adverse weather. By integrating edge computing, they reduce data processing time, leading to faster decision-making. The research demonstrates how intelligent surveillance and AI-driven automation can minimize railway-related accidents and enhance public safety. Additionally, they suggest further improvements, such as incorporating LiDAR sensors and thermal imaging for more advanced detection. Their findings highlight the need for integrating AI with railway infrastructure to create a robust, efficient, and scalable safety solution for unmanned crossings worldwide.

[3] Carvalho et al. (2017), in “Methods to Monitor and Mitigate Wildlife Mortality in Railways,” explore the growing issue of wildlife mortality due to railway infrastructure and propose various mitigation strategies to reduce train-animal collisions. Their study assesses different approaches, including constructing wildlife corridors, implementing fencing, and using AI-powered monitoring systems to detect animal movements near railway tracks. The authors analyze ecological data to determine high-risk areas where train-induced wildlife deaths occur frequently and suggest targeted interventions. They emphasize the importance of interdisciplinary collaboration between ecologists, engineers, and policymakers in developing sustainable railway infrastructure that balances transportation efficiency with biodiversity conservation. By integrating machine learning and remote sensing technologies, such as thermal cameras and GPS-based tracking, real-time monitoring can alert train operators about animal crossings, preventing collisions. Their findings contribute to wildlife protection efforts while ensuring railway safety and operational efficiency. The study also discusses policy recommendations to integrate conservation strategies within railway planning. Future work could explore predictive analytics to anticipate animal

movement patterns and optimize train schedules accordingly. By adopting adaptive and AI-driven solutions, railway authorities can significantly reduce the environmental impact of rail networks while maintaining seamless transportation services.

[4] Malo et al. (2016), in “Railway Poles as Pitfall Traps for Birds,” examine the ecological risks posed by railway poles, which unintentionally trap birds and contribute to avian mortality. Their research investigates how different bird species are affected and identifies specific locations where high casualties occur. The authors emphasize that railway infrastructure must consider wildlife safety to prevent unintended environmental consequences. Their findings indicate that birds often get trapped in hollow poles, leading to preventable deaths. As a solution, they propose structural modifications such as redesigning poles with escape routes and installing physical barriers to prevent entrapment. The study highlights the importance of wildlife-conscious engineering in railway development and calls for policy changes that require bird-safe infrastructure. By integrating simple modifications like covering open-ended poles or adding perches, railways can significantly reduce bird fatalities without disrupting operations. Their research underscores the broader need for incorporating ecological assessments in transportation planning. Future studies could explore new materials and innovative designs that further minimize avian casualties while ensuring railway efficiency. Their work contributes to ongoing conservation efforts, raising awareness about how minor infrastructure changes can have a major impact on protecting bird populations in railway environments.

[5] Lin et al. (2017), in “Focal Loss for Dense Object Detection,” introduce focal loss, a novel function designed to improve the accuracy of object detection models in scenarios with class imbalance. Traditional loss functions often struggle to identify rare or small objects due to overwhelming dominance by easily classified categories. To address this, the authors propose focal loss, which adjusts model training to focus more on difficult-to-classify objects by reducing the weight of well-classified samples. Implemented in RetinaNet, this method significantly enhances detection performance, particularly in dense object detection tasks. Their research demonstrates focal loss’s effectiveness in applications such as autonomous driving, medical imaging, and wildlife monitoring, where detecting rare objects is critical. The study provides a detailed mathematical formulation of focal loss and compares

its performance with existing loss functions. Experimental results show that focal loss substantially improves detection precision, particularly in datasets with a high imbalance. The research has broad implications, influencing advancements in AI-powered object detection across multiple industries. Future work could explore optimizing focal loss for real-time processing and expanding its application to edge computing environments, ensuring faster and more efficient detection capabilities in complex scenarios.

[6] Mamat et al. (2022), in “Animal Intrusion Detection in Farming Area Using YOLOv5 Approach,” develop an AI-based animal intrusion detection system utilizing the YOLOv5 object detection model. Their system processes real-time surveillance footage from strategically placed cameras to identify animals entering restricted farming areas, thereby preventing crop damage and minimizing financial losses for farmers. The model is trained on various animal species, ensuring high detection accuracy in different environmental conditions. Upon detecting an intrusion, the system automatically generates alerts, notifying farmers through mobile applications or automated warning signals, enabling immediate intervention. The study highlights the significance of deep learning in agricultural security, demonstrating how AI-driven automation can enhance farm protection. By integrating IoT-based real-time monitoring with AI-powered object detection, the system ensures effective and continuous surveillance of farming zones. The authors emphasize that their approach helps reduce human-wildlife conflicts, which are a growing concern in rural and semi-urban regions. The research also suggests integrating additional technologies such as thermal imaging and edge computing to improve detection efficiency. Future advancements could focus on predictive analytics to anticipate animal movement patterns and prevent potential intrusions before they occur. Their study provides a scalable and cost-effective solution for improving agricultural security.

[7] Noreen et al. (2017), in “A Study of LoRa Low Power and Wide Area Network Technology,” analyze the capabilities of LoRa (Long Range) technology and its role in various IoT applications. Their research evaluates LoRa’s efficiency in different environments, comparing it with other wireless communication technologies like ZigBee, NB-IoT, and Wi-Fi. The study highlights LoRa’s primary advantages, including low power consumption, long-range connectivity, and superior signal penetration through

obstacles, making it ideal for remote sensing applications. However, the authors also discuss potential challenges, such as interference, limited data rates, and network scalability issues, which must be addressed to improve its widespread adoption. Their findings underscore the importance of LoRa in enabling reliable, energy-efficient, and cost-effective communication for large-scale IoT deployments. Applications of LoRa technology include smart agriculture, industrial IoT, environmental monitoring, and urban infrastructure management. The research suggests that integrating LoRa with AI and edge computing could further optimize network efficiency and enhance its capabilities for real-time data processing. Future developments may focus on improving LoRa's security, reducing latency, and expanding its role in next-generation smart city solutions. Their study provides valuable insights into the evolution of low-power wide-area networks (LPWAN) and their impact on IoT advancements.

[8] Saari et al. (2018), in “LoRa—A Survey of Recent Research Trends,” provide a comprehensive review of LoRa technology, examining its advancements, challenges, and applications in modern IoT networks. Their study explores how LoRa has been optimized to address key issues such as network scalability, security vulnerabilities, and energy efficiency. The authors analyze LoRa's integration with artificial intelligence (AI) and 5G networks, highlighting its potential for future applications in large-scale IoT infrastructures. LoRa's ability to provide long-range, low-power communication makes it a key player in remote sensing, industrial automation, and smart city implementations. The study discusses various enhancements in LoRa's protocols, including adaptive data rate (ADR) mechanisms that improve performance in dense IoT environments. Additionally, the authors evaluate how LoRa's adoption is growing in environmental monitoring, wildlife tracking, and precision agriculture. Their findings suggest that future research should focus on improving LoRa's security protocols and optimizing network efficiency through AI-driven predictive maintenance. By addressing current limitations, LoRa can become a more robust solution for long-distance wireless communication. Their study provides valuable insights into how LoRa continues to evolve, reinforcing its role as an essential technology in the expanding field of IoT connectivity.

[9] Devalal & Karthikeyan (2018), in “LoRa Technology - An Overview,” present a detailed analysis of LoRa technology, focusing on its network architecture, per-

formance characteristics, and practical applications. Their study compares LoRa with alternative communication technologies such as NB-IoT, Sigfox, and LTE-M, highlighting its advantages in power efficiency, long-range connectivity, and cost-effectiveness. The authors examine various real-world use cases where LoRa has been implemented successfully, including smart city solutions, industrial automation, and precision agriculture. Despite its advantages, the study also discusses key challenges such as network congestion, limited data transmission rates, and security vulnerabilities that could hinder LoRa's scalability. The research proposes several future improvements, including adaptive transmission protocols, enhanced encryption methods, and AI-driven network optimization, to make LoRa more efficient and reliable. Their findings emphasize the importance of LoRa in enabling large-scale IoT ecosystems, particularly in remote and rural areas where other communication technologies may be less viable. By addressing these limitations, LoRa can continue to be a major enabler of long-range wireless communication. The study provides a strong foundation for future research aimed at optimizing LoRa's performance and expanding its adoption in diverse IoT applications.

[10] Kellenberger et al. (2017), in "Fast Animal Detection in UAV Images Using Convolutional Neural Networks," introduce a deep learning-based approach for detecting animals in UAV (unmanned aerial vehicle) imagery using convolutional neural networks (CNNs). Their research focuses on automating wildlife monitoring, which traditionally relies on manual image analysis. By leveraging CNN-based object detection models, the system rapidly processes large datasets and accurately identifies animals in aerial surveys, reducing the need for human intervention. The study highlights the effectiveness of AI in conservation efforts, providing real-time population assessments for endangered species and helping researchers track biodiversity changes. The authors emphasize that UAV-based AI technology significantly improves the efficiency of wildlife monitoring, especially in inaccessible regions where traditional ground-based methods are impractical. Their findings demonstrate how integrating UAVs with deep learning can enhance ecological research and species conservation. The study suggests further improvements, such as incorporating multispectral imaging and reinforcement learning algorithms to refine detection accuracy. Future work could explore real-time data transmission via satellite networks, enabling instant updates

for conservation teams. Their research contributes to the growing field of AI-driven environmental monitoring, offering practical solutions for large-scale wildlife tracking and ecosystem management.

[11] G. Chen et al. (2014): This paper investigates the use of deep convolutional neural networks (CNNs) for species recognition in wildlife monitoring. Traditional methods relied on handcrafted features and classical machine learning approaches, which often struggled with variability in animal appearances, lighting conditions, and background noise. The authors propose a CNN-based approach to automatically extract hierarchical features from camera-trap images, significantly improving classification accuracy. The study demonstrates that deep learning models can outperform traditional methods by learning intricate patterns in data without requiring manual feature engineering. The authors trained their CNN model on a dataset of wild animal images, evaluating its performance in identifying different species. Their results show that deep CNNs are highly effective at distinguishing between species, even under challenging conditions. Additionally, they highlight the importance of large-scale labeled datasets to further enhance model performance. By leveraging deep learning, the proposed system enables automated and efficient wildlife monitoring, reducing the reliance on human intervention. This research contributes to the field of ecological conservation by providing a scalable solution for tracking animal populations. The findings have significant implications for biodiversity studies, poaching prevention, and habitat monitoring, demonstrating the potential of AI-driven technologies in wildlife research and environmental protection.

[12] A. Gomez et al. (2016): This study addresses the challenge of species identification in low-quality camera-trap images using very deep convolutional neural networks (CNNs) and confidence thresholds. Camera-trap images are often affected by blurriness, poor lighting, motion artifacts, and occlusions, making automated recognition difficult. Traditional methods struggle in these conditions, leading to a high rate of misclassification. To overcome these issues, the authors propose using deeper CNN architectures that can learn more complex features from images, improving classification accuracy. They introduce confidence thresholds to filter out uncertain predictions, ensuring that only highly reliable classifications are considered. This approach significantly reduces false positives, making it more practical for real-world

deployment in conservation efforts. The authors test their model on a dataset of low-quality wildlife images, demonstrating that deep CNNs can still perform well despite image degradation. Their findings emphasize that confidence-based filtering enhances model reliability, particularly in challenging imaging conditions. The study provides valuable insights into how deep learning can be effectively applied to wildlife monitoring, enabling automated species recognition even when image quality is suboptimal. This research is particularly relevant for conservationists who rely on camera-trap data for tracking and protecting endangered species.

Chapter 3

DESIGN METHODOLOGY

3.1 PREAMBLE

This chapter outlines the architectural design and methodology of the proposed system. It describes the data flow processes to ensure efficient detection and prevention of wildlife-train collisions.

3.2 PROPOSED SYSTEM DESIGN

The proposed system is an early warning mechanism designed to prevent wildlife-train collisions by integrating GSM and CNN technology. It utilizes IR sensors to detect obstacles near railway tracks, triggering a camera to capture images of the detected object. The images are then analyzed using a CNN-based deep learning module to identify whether the obstacle is an animal. Upon identification, a series of actions are initiated, including activating a buzzer to alert nearby personnel, sending notifications to authorities via GSM-based communication, and transmitting data through LoRa to the train unit, where it is displayed to the driver. Additionally, a control signal can activate a circuit breaker relay system to halt train operations in emergencies. The system incorporates components such as NodeMCU for wireless communication, LoRa modules for data transfer, and LCD displays for visual notifications, offering a comprehensive solution to reduce wildlife-train collisions and enhance safety for railway operations.

3.3 BLOCK DIAGRAM

Figure 3.1 and 3.2 shows the working of this system involves multiple steps that ensure real-time monitoring, obstacle detection, and alerting for safety purposes. Here's how it operates.

1. Transmitter Unit (Detection and Alert Transmission) The Transmitter Unit is responsible for detecting an animal's presence, capturing data, and transmitting alerts. The NodeMCU (ESP8266/ESP32) acts as the primary controller, coordinating all sensors and communication modules.

Component Explanations:

- **IR Sensor:** The IR sensor detects movement when an animal passes in front of it. Once an animal is detected, it triggers further actions in the system.
- **Camera:** After the IR sensor detects an animal, the camera captures an image or video of the detected object for further identification. This can help in classifying the animal type using machine learning models.
- **Laptop:** The laptop is used for processing captured images and training detection models, enabling the system to identify specific animals. It can also be used for monitoring live data.
- **GPS Module:** When an animal is detected, the GPS module retrieves the precise geographical location of the detection site. This location is sent along with the alert message for tracking purposes.
- **GSM Module:** The GSM module sends an SMS alert to a registered mobile phone with the detection details, including the type of detected animal and its GPS location. This ensures that the user receives real-time notifications even if they are far from the system.
- **LoRa Tx (Transmitter):** LoRa (Long Range) communication is used to transmit detection data wirelessly to a remote receiver unit. LoRa technology enables long-range communication with low power consumption.

Transmitter Unit

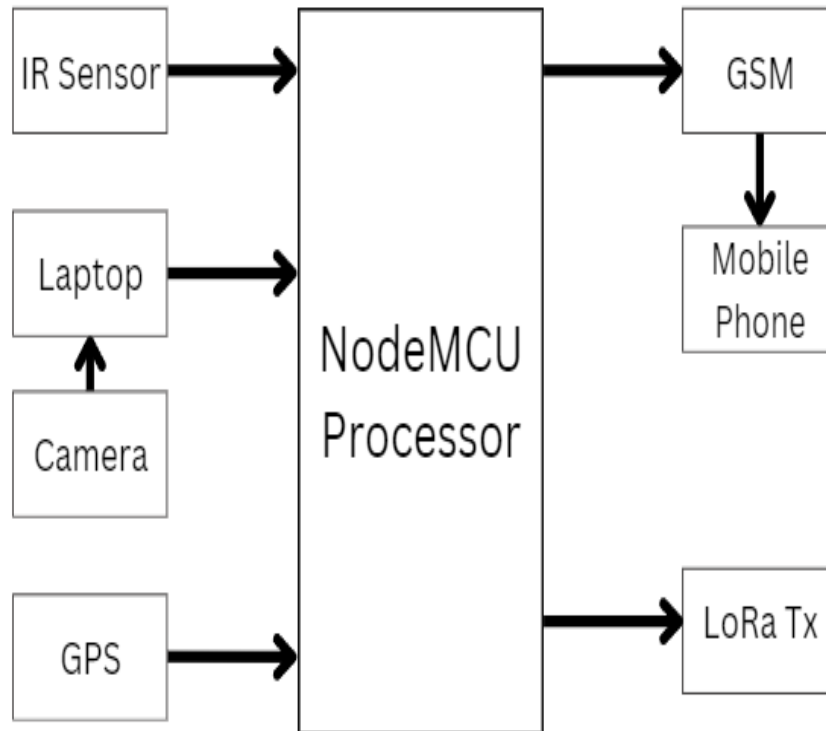


Figure 3.1: Block diagram of the transmitter

2. Receiver Unit (Alert Processing and Display) The Receiver Unit processes the transmitted data and provides alerts to the user. An Arduino Nano acts as the central processor for this unit.

Component Explanations:

- **LoRa Rx (Receiver):** This module receives the transmitted data from the LoRa Tx module in the Transmitter Unit. The received data contains information about the detected animal and its location.
- **Arduino Nano:** The Arduino Nano processes the received data and controls the output components of the receiver unit, such as the LCD display and buzzer.

- **LCD Display:** The LCD screen displays the details of the detected animal, including its identification (if available) and GPS location. This allows the user to take necessary actions.
- **Buzzer:** The buzzer in the receiver unit provides an additional sound alert, notifying the user about the animal's presence.

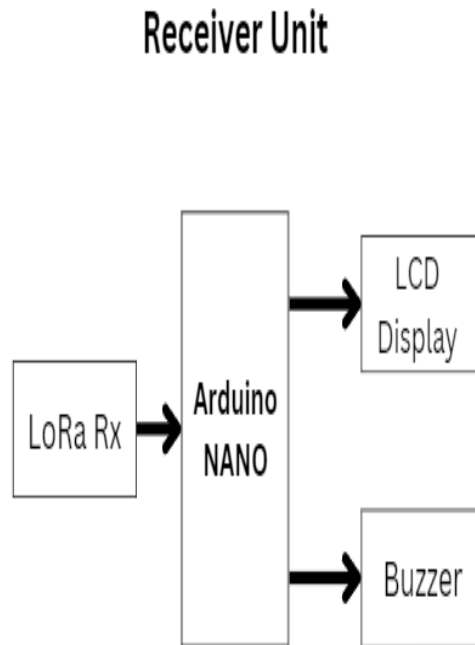


Figure 3.2: Block diagram of the receiver

3.4 FLOW CHART

The flowchart in figure 3.3 represents an automated animal and human detection system designed to enhance railway safety by identifying potential obstacles using sensors, image processing, and communication technologies. The system initiates its operation by initializing key components, including sensors, and camera. These components work together to monitor the environment, ensuring real-time detection of any movement near the railway tracks. The primary function of the system is to check for movement using an infrared (IR) sensor, which continuously scans the surroundings.

If no motion is detected, the system remains in an idle loop, persistently monitoring the area. However, when the IR sensor detects movement, the system transitions to the next stage by capturing and sending data to a laptop for further processing. Once the data reaches the laptop, image processing techniques analyze the captured visuals to determine whether the detected object is an animal or a person. If the system finds no valid detection, it re-enters the monitoring loop, continuing its surveillance process. However, if an animal or human is identified, the system activates additional components to ensure an effective response. The GPS module retrieves the location coordinates of the detection, while a GSM module transmits this information as an alert. This real-time messaging ensures that railway authorities and relevant personnel are immediately informed about the detected presence near the tracks.

Simultaneously, the system transmits data using a LoRa module, enabling long-range communication. This ensures that even if the railway track is in a remote location, the detection alert reaches the appropriate authorities. The receiver module installed on the train unit plays a crucial role in receiving this transmitted data. If the receiver module successfully obtains the alert, the detected information is displayed on an LCD screen, providing immediate visibility to train operators. This allows train operators to take necessary action, such as slowing down or stopping the train, to prevent accidents.

Once the information is displayed and the necessary alerts are sent, the system resets itself and re-enters the monitoring phase. This continuous cycle ensures round-the-clock surveillance, reducing the likelihood of collisions and enhancing railway

safety. By automating the detection and alert system, this technology minimizes human intervention, providing a reliable and efficient solution for preventing accidents involving animals and humans on railway tracks.

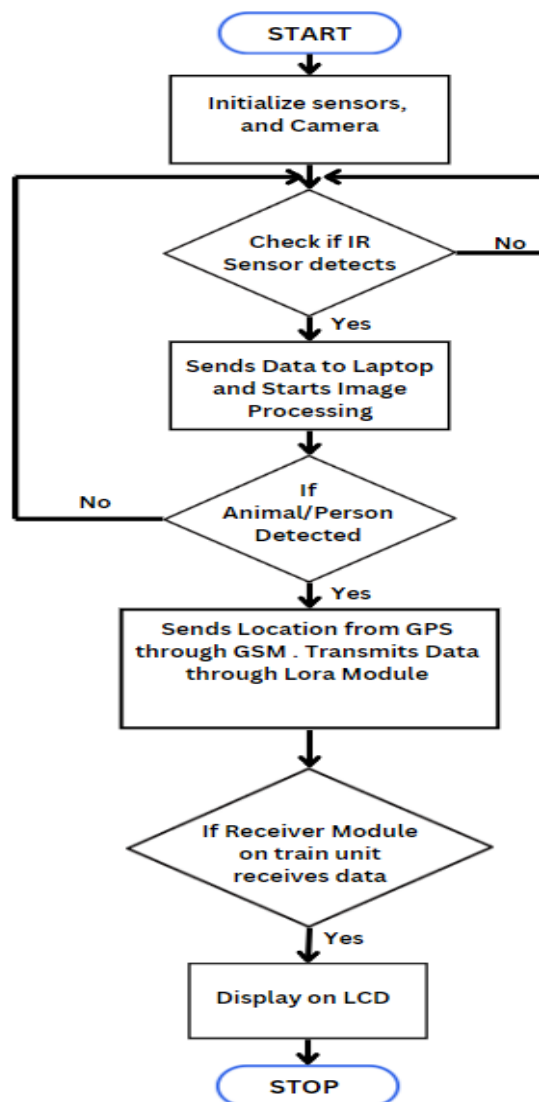


Figure 3.3: Flow chart of the system

3.5 CIRCUIT DIAGRAM

The figures shown below shows the circuit diagram of the proposed system.

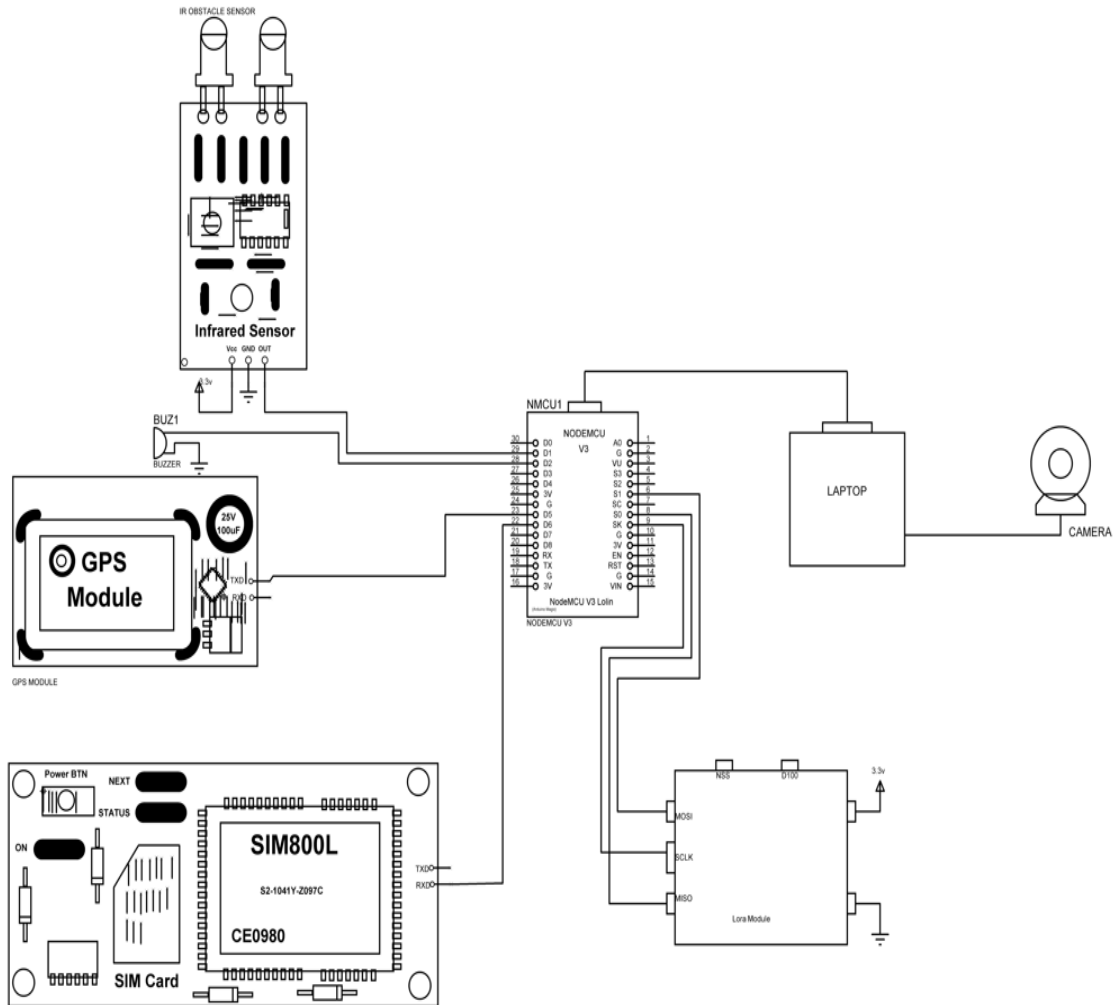


Figure 3.4: Circuit diagram of transmitter

The diagram in figure 3.4 represents an early warning system integrating multiple modules for obstacle detection, GPS tracking, GSM-based alerting, and long-range communication using LoRa. At the core of the system is the NodeMCU (ESP8266) microcontroller, which connects and controls different components. An IR obstacle sensor detects objects and sends signals to the NodeMCU, which can then trigger a buzzer as an immediate alert. The GPS module is used to obtain location data, which can be sent via the SIM800L GSM module to a remote user through SMS.

Additionally, a LoRa module enables long-range wireless communication, possibly for transmitting alerts or data to a central server or another LoRa-enabled device. The laptop and camera are likely used for real-time monitoring, possibly assisting in image processing or remote visualization. The entire system operates at 3.3V, with proper connections for power and ground. This setup is designed for real-time obstacle detection and warning, making it suitable for applications such as wildlife monitoring, security surveillance, or accident prevention.

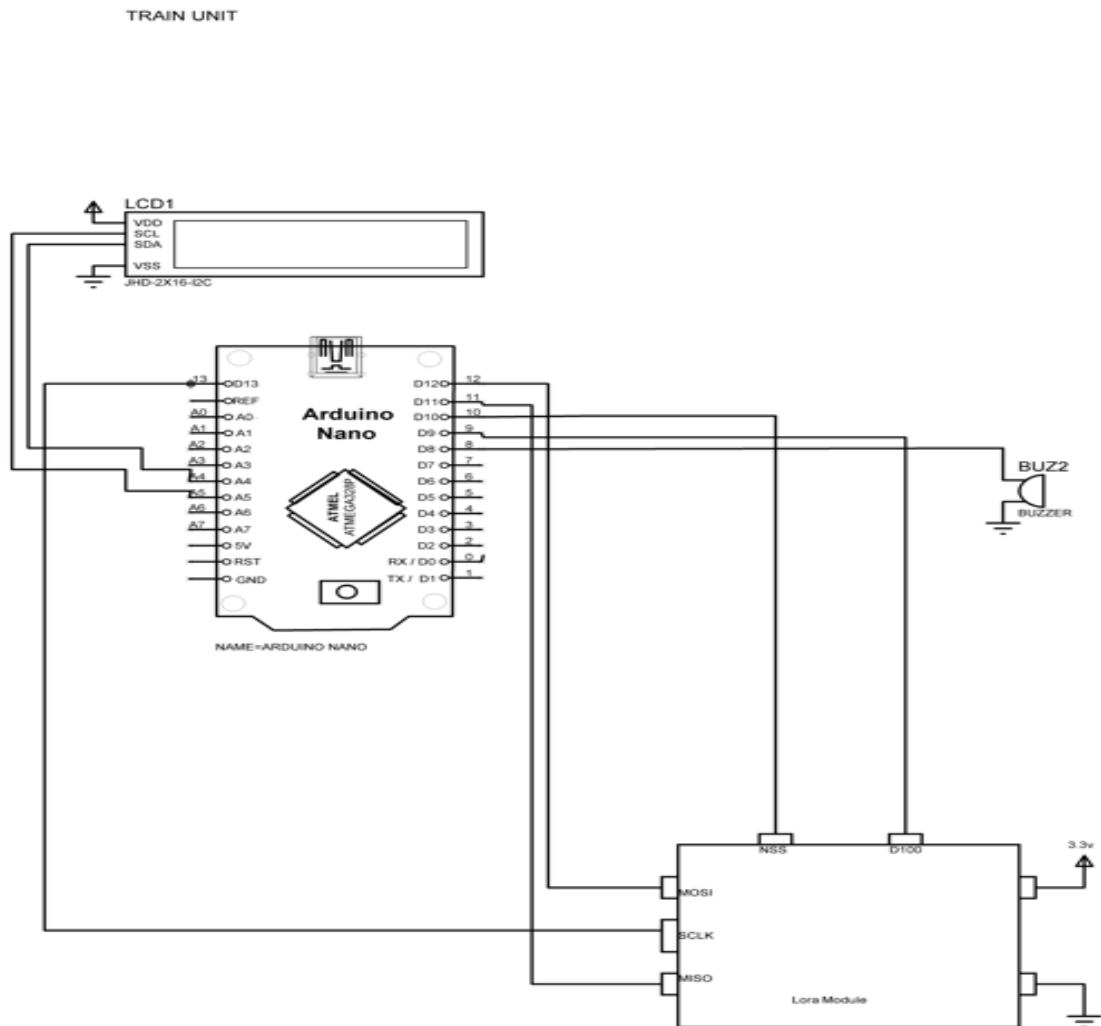


Figure 3.5: Circuit diagram of receiver

The diagram in figure 3.5 represents a train unit controlled by an Arduino Nano, which is responsible for processing and managing communication and alerts. The system includes an LCD display (3HD-2X16-6C) connected to the Arduino via the

SCL and SDA lines, which suggests it operates over the I2C protocol to display real-time data. Additionally, a LoRa module is integrated, connected via SPI (MOSI, MISO, SCLK, NSS, and DIO0) to enable long-range wireless communication. This allows the train unit to send and receive information over significant distances, which can be useful for monitoring and control. A buzzer is included, likely to provide audio alerts for critical warnings or notifications. The entire system is powered at 3.3V, which aligns with the operating voltage of the LoRa module. This setup suggests that the train unit is part of a larger communication network, possibly for railway safety, real-time tracking, or collision avoidance systems.

3.6 SUMMARY

The early warning system integrates modules for obstacle detection, GPS tracking, GSM-based alerting, and long-range LoRa communication, with the NodeMCU (ESP8266) as the central controller. An IR obstacle sensor detects objects, triggering a buzzer for immediate alerts. The GPS module provides location data, which the SIM800L GSM module can transmit via SMS. A LoRa module enables wireless communication, while a laptop and camera assist in real-time monitoring and image processing. Operating at 3.3V, this system is suitable for wildlife monitoring, security, and accident prevention.

The train unit, controlled by an Arduino Nano, manages communication and alerts. It features an LCD display connected via I2C for real-time data output and a LoRa module via SPI for long-range communication. A buzzer provides critical warnings. Running on 3.3V, this system is likely part of a railway safety network, enabling real-time tracking and collision avoidance.

Chapter 4

SYSTEM DEVELOPMENT

This chapter provides a detailed exploration of the development of the proposed early warning system designed to prevent elephant-train collisions. The chapter is divided into two primary sections: hardware and software development.

The hardware section discusses the components used, including microcontrollers like NodeMCU and Arduino Nano, communication modules such as the RA-02 LoRa module, and various sensors and display units essential for real-time detection and alert systems. Additionally, the integration of a GPS module and a GSM module enhances the system's ability to provide location tracking and emergency alerts.

The software section details the programming environments and tools utilized, such as the Arduino IDE, Anaconda Navigator, and Spyder, highlighting their roles in achieving seamless integration of the hardware components and ensuring effective system functionality. It also discusses the implementation of machine learning techniques for object detection and classification, further improving the system's efficiency.

This chapter serves as the foundation for understanding the implementation of the project, offering insights into the interaction between its hardware and software components, ensuring a robust and reliable solution for minimizing elephant-train collisions.

4.1 HARDWARE

4.1.1 NODEMCU PROCESSOR

The NodeMCU is an open-source IoT development board that integrates the ESP8266 Wi-Fi module with a simple-to-use Lua-based firmware. Designed for Internet of Things (IoT) applications, it is widely used due to its low cost, Wi-Fi capability, and ease of programming. The ESP8266 chip, manufactured by Espressif Systems, provides built-in Wi-Fi support, making it ideal for wireless communication in smart home automation, industrial monitoring, and other IoT applications.

The board comes with a USB-to-serial adapter (CP2102 or CH340) for easy programming via a micro-USB cable. NodeMCU features multiple GPIO (General Purpose Input/Output) pins, I2C, SPI, UART communication protocols, and PWM support, allowing developers to interface it with various sensors, actuators, and external devices. Additionally, it has an ADC (Analog-to-Digital Converter) pin, enabling it to read analog signals.

One of its biggest advantages is compatibility with the Arduino IDE, which allows programming in C/C++ alongside the native Lua scripting language. This makes it accessible to both beginners and advanced developers. The built-in Wi-Fi is a key feature that enables NodeMCU to connect with cloud platforms such as Blynk, Firebase, Thingspeak, and MQTT brokers, facilitating remote data monitoring and control. It supports both AP (Access Point) and Station mode, allowing it to either create a Wi-Fi network or connect to an existing one.

NodeMCU operates on 3.3V logic levels, meaning peripherals should be voltage-matched to prevent damage. Powering the board can be done via the micro-USB port or through the Vin pin, which can accept a 5V input. Its small form factor and low power consumption make it ideal for battery-powered IoT applications. Moreover, the deep sleep mode significantly reduces power usage, making it useful for energy-efficient projects.

The programming environment for NodeMCU includes Arduino IDE, PlatformIO, and the Lua-based ESPlorer IDE. The ESP8266 core for Arduino extends compatibility, providing a wide range of libraries for interfacing with sensors, displays, and cloud

services. It is also possible to flash the board with MicroPython, an alternative to Lua that provides Python scripting capabilities for embedded development.

NodeMCU's open-source nature has led to an active developer community, contributing extensive documentation, libraries, and support forums. Common applications include home automation (e.g., smart lights, switches, thermostats), weather monitoring stations, wireless sensor networks, robotics, and industrial IoT. It can communicate with devices over HTTP, MQTT, or WebSockets, making it suitable for real-time data transfer in IoT ecosystems.

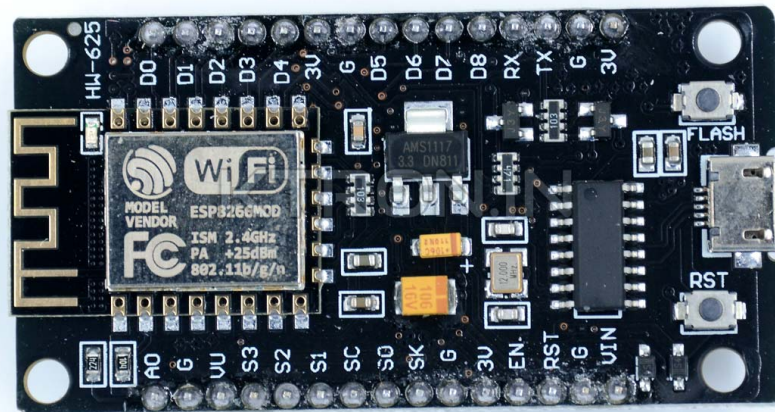


Figure 4.1: NodeMCU processor

Security considerations include using encryption (TLS/SSL) for data transmission and securing access points to prevent unauthorized control. NodeMCU has evolved with newer versions, including ESP8266-12E and ESP8266-12F, offering better stability and extended GPIO options. For more advanced projects requiring higher processing power and additional features like Bluetooth, ESP32-based NodeMCU

boards serve as a powerful upgrade. NodeMCU is a versatile, affordable, and beginner-friendly microcontroller board with built-in Wi-Fi, making it an excellent choice for IoT and embedded systems development. Its combination of ease of programming, wireless capability, and low power consumption ensures its continued popularity in the maker community and professional IoT solutions.

FEATURES

- 802.11 b/g/n support
- 802.11 n support(2.4 GHz), up to 72.2 Mbps
- Defragmentation
- 2 x virtual Wi-Fi interface
- Automatic beacon monitoring (hardware TSF)
- Infrastructure BSS Station mode/ SoftAP mode/ Promiscuous mode

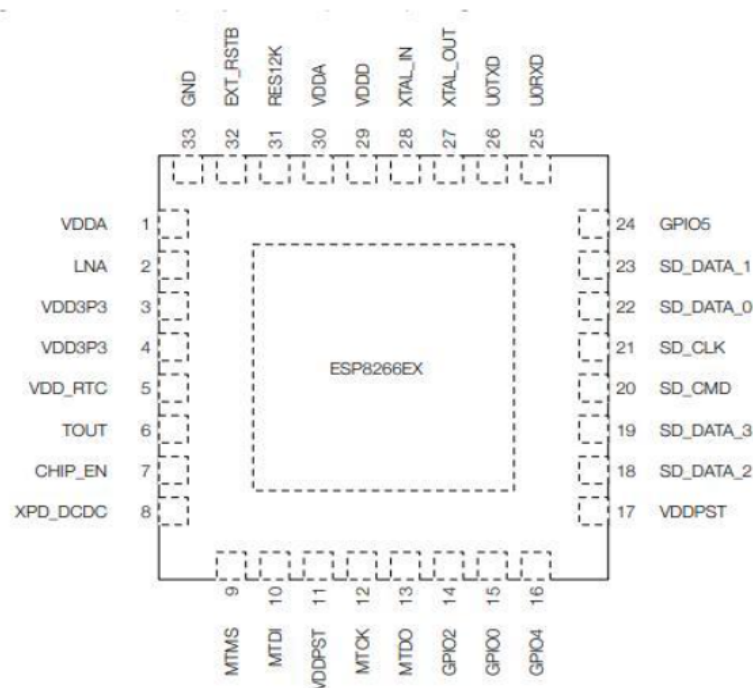


Figure 4.2: NodeMCU pinout

Table 4.1: NodeMCU Pinout

PINOUT OF NODEMCU			
Pins	Name	Type	Function
1	VDDA	P	Analog Power 2.5V-3.6V
2	LNA	I/O	RF antenna interface.Chip output impedance= 39+j6 Ohm .
3	VDD3P3	P	Amplifier Power 2.5V-3.6V
4	VDD3P3	P	Amplifier Power 2.5V-3.6V
5	VDD_RTC	P	NC (1.1V)
6	TOUT	I	ADC Pin.It can be used to test the power supply voltage of VDD3P3(Pin 3 and pin 4) and the input power voltage of TOUT (Pin 6)
7	CHIP_EN	I	Chip Enable
8	XPD_DCDC	I/O	Deep sleep wakeup: GPIO16
9	MTMS	I/O	GPIO14:HSPI_CLK
10	MTDI	I/O	GPIO12:HSPI_MISO
11	VDDPST	P	Digital/IO Power Supply(1.8V-3.6V)
12	MTCK	I/O	GPIO13:HSPI_MOSI;UART0_CTS
13	MTDO	I/O	GPIO15:HSPI_CS; UART0_RTS
14	GPIO2	I/O	UART TX during flash programming
15	GPIO0	I/O	SPIO_CS2
16	GPIO4	I/O	GPIO4
17	VDDPST	P	Digital I/O power supply(1.8V-3.6V)
18	SDIO_DATA2	I/O	Connect to SD_D2
19	SDIO_DATA_3	I/O	Connect to SD_D3
20	SDIO_CMD	I/O	Connect to SD_CMD
21	SDIO_CLK	I/O	Connect to SD_CLK
22	SDIO_DATA_0	I/O	Connect to SD_D0
23	SDIO_DATA_1	I/O	Connect to SD_D1
24	GPIO5	I/O	GPIO5
25	U0RXD	I/O	UART Rx during flash programming
26	U0TXD	I/O	UART Tx during flash programming
27	XTAL_OUT	I/O	Connect to crystal oscillator output
28	XTAL_IN	I/O	Connect to crystal oscillator input
29	VDDD	P	Analog Power 2.5V-3.6V
30	VDDA	P	Analog Power 2.5V-3.6V
31	RES12K	I	Serial Communication with a 12KOhm
32	EXT_RSTB	I	External reset signal

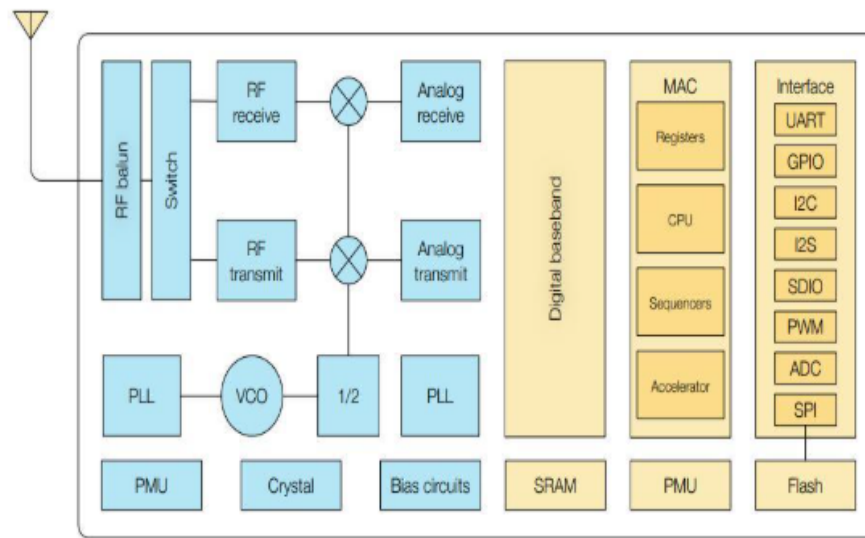


Figure 4.3: Function blockdiagram of NodeMCU

The ESP8266 uses a 32bit processor with 16 bit instructions. It is Harvard architecture which mostly means that instruction memory and data memory are completely separate. The ESP8266 has on die program Read-Only Memory (ROM) which includes some library code and a first stage boot loader. All the rest of the code must be stored in external Serial flash memory (provides only serial access to the data rather than addressing individual bytes, the user reads or writes large contiguous groups of bytes in the address space serially). Depending on your ESP8266, the amount of available flash memory can vary.

As any other microcontroller, ESP8266 has a set of GPIO pins (General Purpose Input(Output pins) that we can use to “control” external sensors. Our ESP8266 has 17 GPIO pins but only 11 can be used (among 17 pins, 6 are used for communication with the on-board flash memory chip). It also has an analog input (to convert a voltage level into a digital value that can be stored and processed in the ESP8266). It also has a WIFI communication to connect your ESP8266 to your WIFI network, connect to the internet, host a web server, let your smartphone connect to it, etc.

Another advantage of an ESP8266 is that it can be programmed as any other microcontroller and especially any Arduino. The provided block diagram represents the architecture of a wireless communication system, likely a System-on-Chip (SoC) for RF (Radio Frequency) communication. The design consists of multiple functional

units, each responsible for different aspects of signal processing and data handling.

The RF front-end includes an antenna, an RF balun, a switch, and RF receive/transmit chains. These components handle the wireless signal transmission and reception, converting signals between RF and baseband frequencies. The frequency synthesis section consists of Phase-Locked Loops (PLLs), a Voltage-Controlled Oscillator (VCO), a crystal oscillator, and bias circuits. These ensure stable clock generation and frequency accuracy for RF communication.

The analog baseband section processes incoming and outgoing signals before digital conversion. The digital baseband handles key tasks like data processing, modulation, demodulation, and encoding. Memory components, including SRAM and Flash, store temporary and permanent data, such as firmware. The Media Access Control (MAC) unit includes the CPU, registers, sequencers, and accelerators, which manage communication protocols.

The interface section offers multiple connectivity options such as UART, GPIO, I2C, I2S, SDIO, PWM, ADC, and SPI, ensuring compatibility with external devices. The Power Management Unit (PMU) regulates power consumption for efficiency.

This architecture is common in wireless communication chips, including Wi-Fi, Bluetooth, and IoT devices, enabling seamless data transmission and processing.

Table 4.2: Technical specifications of NodeMCU module

Categories	Items	Parameters
Wi-Fi	Certification	Wi-Fi Alliance
	Protocols	802.11 b/g/n (HT20)
	Frequency Range	2.4 GHz – 2.5 GHz (2400 MHz – 2483.5 MHz)
	TX Power	802.11 b: +20 dBm, 802.11 g: +17 dBm, 802.11 n: +14 dBm
	Rx Sensitivity	802.11 b: -91 dBm (11 Mbps), 802.11 g: -75 dBm (54 Mbps), 802.11 n: -72 dBm (MCS7)
Hardware	Antenna	PCB Trace, External, IPEX Connector, Ceramic Chip
	CPU	Tensilica L106 32-bit processor
	Peripheral Interface	UART, SDIO, SPI, I2C, I2S, IR Remote Control, GPIO, ADC, PWM, LED Light, Button
	Operating Voltage	2.5 V – 3.6 V
	Operating Current	Average value: 80 mA
	Operating Temperature Range	-40°C – 125°C
Software	Package Size	QFN32-pin (5 mm × 5 mm)
	Wi-Fi Mode	Station/SoftAP/SoftAP+Station
	Security	WPA/WPA2
	Encryption	WEP, TKIP, AES
	Firmware Upgrade	UART Download, OTA (via network)
	Software Development	Supports Cloud Server Development, Firmware, and SDK for fast on-chip programming
	Network Protocols	IPv4, TCP, UDP, HTTP
	User Configuration	AT Instruction Set, Cloud Server, Android/iOS App

4.1.2 ARDUINO NANO PROCESSOR

The Arduino Nano is a compact, versatile microcontroller board designed for embedded systems and DIY electronics projects. It is based on the ATmega328P microcontroller, the same chip used in the Arduino Uno, but in a much smaller form factor. Measuring only 45mm x 18mm, the Nano is ideal for space-constrained applications, making it popular among hobbyists, students, and engineers alike.

Unlike the Uno, the Nano does not come with a standard DC power jack; instead, it features a Mini-USB port for programming and power, supporting 5V logic levels. It can be powered via USB, a 7-12V unregulated power source through the VIN pin, or a 5V regulated power supply through the 5V pin. The board includes 14 digital I/O pins, of which 6 support PWM (Pulse Width Modulation), and 8 analog input pins, providing flexibility for interfacing with a wide range of sensors, actuators, and other peripherals.

It also includes support for I2C, SPI, and UART (serial communication), enabling

communication with other microcontrollers, sensors, and modules such as OLED displays, motor drivers, or wireless transceivers. The Arduino Nano is fully compatible with the Arduino IDE (Integrated Development Environment), allowing users to write, upload, and debug code using C and C++ with Arduino's simplified programming framework. Additionally, it supports various libraries for interfacing with components like LCDs, servos, and sensors, reducing development time for projects.

One of the most significant advantages of the Arduino Nano is its small size and breadboard-friendly design, making it perfect for prototyping compact electronic circuits without the need for soldering. It also has an onboard 16MHz crystal oscillator, ensuring reliable timing for real-time applications. The board features an auto-reset function, eliminating the need for manual resets during programming.

Unlike the Arduino Uno, which uses a removable DIP package microcontroller, the Nano's ATmega328P is soldered directly onto the board, which saves space but makes it harder to replace in case of failure. Additionally, some versions of the Nano use the ATmega328P-AU variant, which has the same functionality but comes in a surface-mount package.

The Arduino Nano comes in different versions, including the original Nano from Arduino and several third-party clones that use different USB-to-serial chips, such as the CH340 instead of the FT232RL, which may require additional drivers for proper communication with a computer. Compared to other microcontrollers, such as the Arduino Pro Mini, the Nano provides a built-in USB interface, making programming easier without the need for an external USB-to-serial adapter.

It is widely used in robotics, home automation, IoT (Internet of Things), wearable technology, data logging, and sensor-based systems due to its power efficiency and ease of use. Despite its small size, the Arduino Nano is highly capable of handling complex tasks like signal processing, motor control, and real-time data acquisition when combined with appropriate external components. With its vast community support and open-source nature, the Arduino Nano remains a go-to choice for beginners and advanced users alike who need a compact, affordable, and powerful microcontroller for their electronic projects.

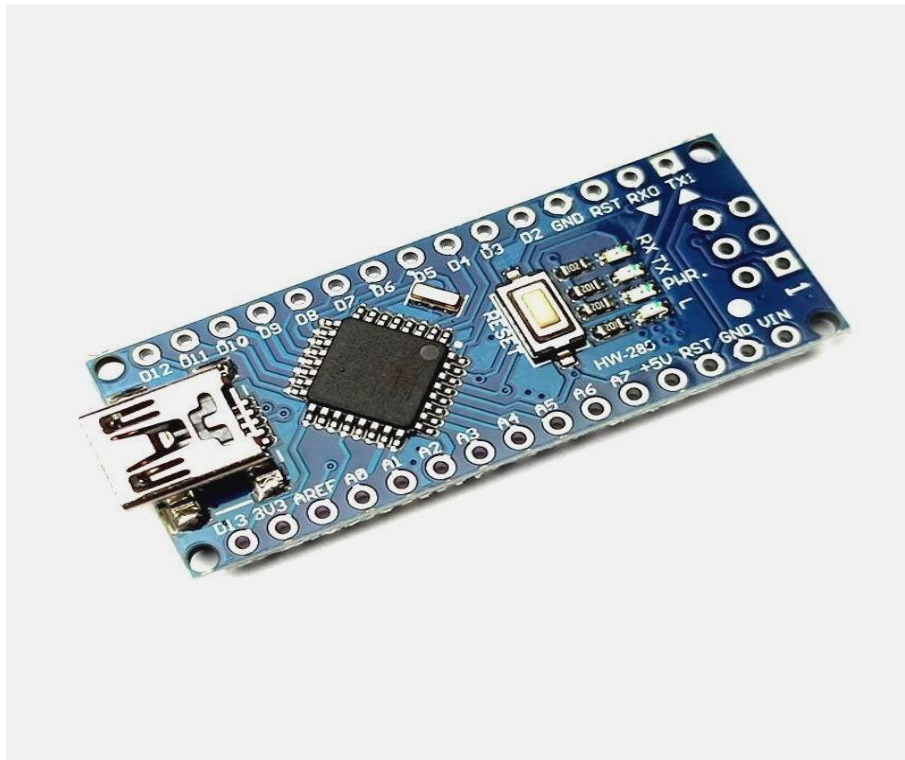


Figure 4.4: Arduino NANO processor

Arduino Nano Communication

The communication of an Arduino Nano board can be done using different sources like using an additional Arduino board, a computer, otherwise using microcontrollers. The microcontroller using in Nano board (ATmega328) offers serial communication (UART TTL). This can be accessible at digital pins like TX, and RX. The Arduino software comprises of a serial monitor to allow easy textual information to transmit and receive from the board. The TX & RX LEDs on the Nano board will blink whenever information is being sent out through the FTDI & USB link in the direction of the computer. The library-like SoftwareSerial allows serial communication on any of the digital pins on the board. The microcontroller also supports SPI & I2C (TWI) communication.

Features

- ATmega328P Microcontroller is from 8-bit AVR family
- Operating voltage is 5V

- Input voltage (V_{in}) is 7V to 12V
- Input/Output Pins are 22
- Analog I/O pins are 6 from A0 to A5
- Digital pins are 14
- Power consumption is 19 mA
- I/O pins DC Current is 40 mA
- Flash memory is 32 KB
- SRAM is 2 KB
- EEPROM is 1 KB
- CLK speed is 16 MHz
- Supports three communications like SPI, IIC, & USART

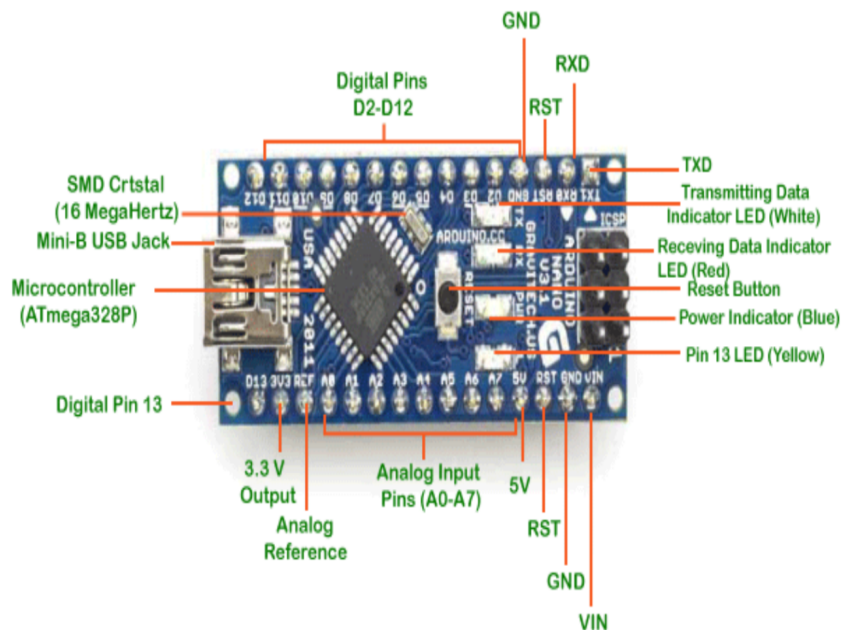


Figure 4.5: Arduino NANO pinout

The board includes the mini-USB jack to connect computer and the Nano board. The DC power jack is absent in Arduino Nano board.

- **Atmega328P Microcontroller:** It is a high performance and low powered 8-bit microcontroller, which is based on AVR RISC Architecture. Here, AVR stands for Audio Video Recorder and RISC stands for Reduced Instruction Set Computing. It is also considered as the most popular AVR controller. It consumes less power than Atmega328 Microcontroller.
- **Serial Pins (Tx, Rx):** TXD and RXD pins are used for serial communication. The TXD is used for transmitting the data, and RXD is used for receiving the data. It also represents the successful flow of data from computer to the board.
- **Mini USB:** It is smaller than the standard USB but thick than the micro-USB. It allows the board to connect to the computer. It is essential for the programming of the Arduino Nano board. Usually plug the USB cable in the Mini-USB jack to load the sketch to the board.
- **SMD Crystal:** It have better stability than other crystals. The device can be easily soldered on any board. The SMD crystal used in the Arduino Nano is of 16MHz.
- **RST(Reset):** It is used to add a Reset button to the connection.
- **Power Indicator:** It indicates the status of the battery. It can also show the voltage of the battery on the LCD display connected to the Arduino board.
- **External Interrupts (2, 3):** These pins are used to activate an interrupt.
- **Digital Pins (D0 – D13):** There are 14 digital I/O pins. The six pins from the set of digital pins are PWM (Pulse Width Modulation) pins numbered D3, D5, D6, D9, D10, and D11. The digital pins have the value either HIGH or LOW.
- **Analog Pins(A0-A7):** There are eight analog pins numbered from A0 to A7. The function of Analog pins is to read the value of analog sensor used in the connection. It can also act as GPIO (General Purpose Input Output) pins.
- **Analog Reference or (AREF):** The AREF pin acts as a reference voltage to feed the Arduino from an external power supply voltage.

- **SPI (10, 11, 12, & 13):** These pins are used for supporting SPI communication.
- **Inbuilt LED (13):** This pin is used to activate the LED.
- **IIC (A4, A5):** These pins are used for supporting TWI communication.
- **Vin:** It is defined as the input voltage, which is applied to the Arduino Board when it is using an external power source.
- **3V3:** The 3V3 pin works as the output regulated voltage of 3.3V.
- **5V:** The 5V pin works as the output regulated voltage of 5V. The power source of 5V for the Arduino Nano board are USB connector, DC power jack, and the Vin. The power can be supplied to the board from either of the above specified sources

4.1.3 LORA RA-02

LoRa RA-02 is a high-performance wireless module based on the SX1278 chip, designed for long-range, low-power communication in IoT applications. It operates in the 433MHz frequency band and utilizes LoRa (Long Range) modulation, enabling it to transmit data over distances of up to 10 kilometers in open areas while maintaining minimal power consumption. This makes it ideal for applications such as smart agriculture, industrial automation, environmental monitoring, and asset tracking. One of the key features of LoRa RA-02 is its ability to achieve robust communication even in environments with high interference, thanks to its spread spectrum technology and strong signal penetration capabilities. It supports a maximum transmission power of 20dBm (100mW) and a sensitivity of -148dBm, allowing it to maintain connectivity even in challenging conditions. Additionally, it features low power consumption in standby and sleep modes, making it an excellent choice for battery-operated devices. The module supports SPI (Serial Peripheral Interface) communication, allowing seamless integration with microcontrollers like Arduino, ESP32, and STM32. Configuring the RA-02 requires setting parameters such as frequency, bandwidth, spreading factor, and coding rate, which influence the trade-off between range, data rate, and power consumption.

While LoRa technology is not suitable for high-speed data transmission, its strength lies in its ability to support long-range communication with low data rates, making it ideal for applications where periodic small data transmissions are sufficient. The RA-02 module also enables LoRaWAN implementation, which is a network protocol built on LoRa technology that allows multiple devices to communicate over a wide-area network with centralized management. However, to use LoRaWAN, additional configuration and a compatible gateway are required. The module's compact size (17 x 16mm) and onboard antenna interface make it easy to integrate into various IoT devices. It is essential to note that the module requires an external antenna for optimal performance, and improper antenna selection or placement can significantly impact signal range. Proper heat dissipation should also be considered to ensure stable operation during prolonged transmission. While the LoRa RA-02 module is an excellent choice for long-range wireless applications, developers must comply with local regulatory requirements regarding frequency use and transmission power limitations.



Figure 4.6: LoRa module

Table 4.3: LoRa pinout

LORA PINS		
Pin	Name	Description
1	GND	Ground (0V)
2	D101	Digital I/O
3	D102	Digital I/O
4	D103	Digital I/O
5	VCC	Power (3.6V maximum)
6	MISO	SPI Data Output)
7	MOSI	SPI Data Input
8	SLCK	SPI Clock
9	NSS	SPI Chip Select
10	D100	Digital I/O
11	RESET	Reset
12	GND	Ground (0V)

4.1.4 JHD162A: 2x16 ALPHANUMERIC LCD MODULE

The JHD162A is a 2x16 alphanumeric LCD module widely used in embedded systems, industrial automation, and electronic projects for displaying text-based information. Featuring a 16-character, 2-line display, this module is based on the Hitachi HD44780 LCD controller, making it compatible with most microcontrollers, including Arduino, Raspberry Pi, PIC, AVR, and STM32. The display operates on a 5V power supply and features an LED backlight, which enhances visibility in low-light conditions. It supports both 4-bit and 8-bit parallel data interfaces, giving developers flexibility in how they connect and control the display. With a 5x8 dot matrix per character format, the JHD162A ensures clear and readable text output. The module includes 16 pins, where key connections include VCC (Power), GND (Ground), RS (Register Select), RW (Read/Write), E (Enable), and D0–D7 (Data Lines). In 4-bit mode, only D4 to D7 are used, reducing the number of required microcontroller pins. The contrast of the LCD can be adjusted using a potentiometer connected to the V0 pin, allowing customization based on environmental lighting conditions.



Figure 4.7: LCD display

One of the key advantages of the JHD162A is its built-in character generator ROM (CGROM), which stores a set of predefined characters, including English letters, numbers, and special symbols. Additionally, it supports custom characters via the Character Generator RAM (CGRAM), enabling developers to create icons, symbols, or non-standard fonts. The module's command set allows operations like clearing the display, cursor movement, blinking text, and shifting text left or right, making it a powerful tool for interactive applications. The JHD162A is commonly used in applications such as DIY electronics, industrial instrumentation, home automation, medical devices, and vending machines, where reliable text output is required. Its simple interface, low power consumption, and cost-effectiveness make it a preferred choice for hobbyists and professionals alike. Programming the LCD with Arduino is straightforward using the LiquidCrystal library, which simplifies text display and cursor control. The display refresh rate is sufficient for static and moderately dynamic text-based applications, although it is not suitable for graphical or high-speed visual updates.

The module operates reliably between 0°C to 50°C, making it suitable for most indoor applications but not ideal for extreme environmental conditions. Compared to

OLED or TFT displays, the JHD162A lacks graphical capabilities but excels in low-power consumption and ease of use. When interfacing with I2C modules, an I2C adapter (PCF8574) can be used to reduce the number of required microcontroller pins, making it more efficient for applications where GPIO pins are limited. The JHD162A is widely available, cost-effective, and well-documented, making it a popular choice in educational and professional projects. While newer display technologies like OLED and e-paper offer higher contrast and wider viewing angles, the JHD162A remains a reliable and budget-friendly option for text-based information displays, making it an excellent choice for a wide range of embedded system applications.

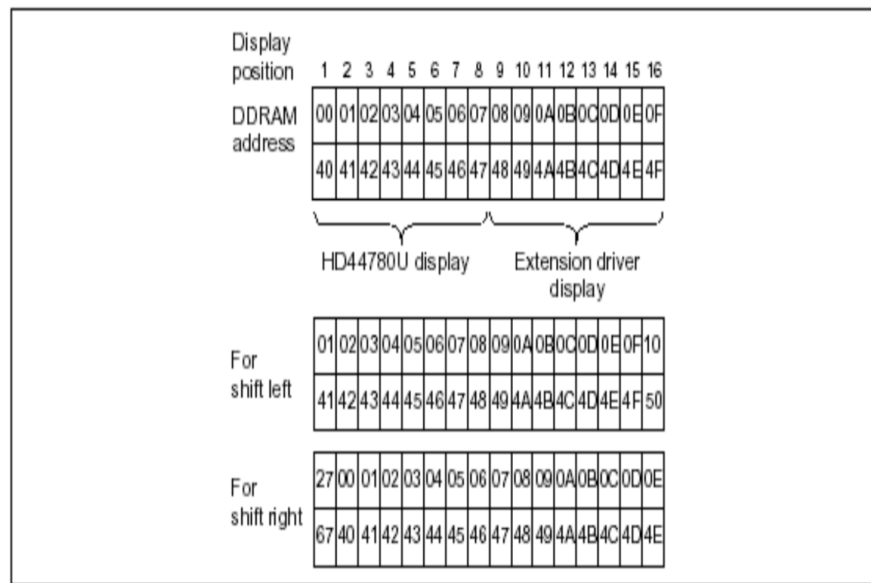


Figure 4.8: 2-line by 16-character display

Features

- 5 × 8 dot matrix possible
- Low power operation support: 2.7 to 5.5V
- Wide range of liquid crystal display driver power: 3.0 to 11V
- Liquid crystal drive waveform
- A One line frequency AC waveform

- Correspond to high speed MPU bus interface
- 4-bit or 8-bit MPU interface enabled
- 80×8 -bit display RAM (80 characters max.)
- 9,920-bit character generator ROM for a total of 240 character fonts
- 208 character fonts (5×8 dot)
- 32 character fonts (5×10 dot)
- 16-common $\times$ 40-segment liquid crystal display driver
- Programmable duty cycles
- 1/8 for one line of 5×8 dots with cursor
- 1/16 for two lines of 5×8 dots with cursor
- Wide range of instruction functions:
- Display clear, cursor home, display on/off, cursor on/off, display character blink, cursor shift, display shift
- 61 x 15.8 mm viewing area
- TTL and CMOS compatible

4.1.5 IR MODULE

Infrared (IR) sensors are versatile electronic devices used to detect infrared radiation emitted or reflected by objects. Operating on the principles of infrared light detection, these sensors are commonly employed in various applications, including motion detection, object tracking, proximity sensing, and temperature measurement. IR sensors are categorized into two primary types: active and passive. Active IR sensors emit infrared radiation from an LED and detect reflections from nearby objects, making them ideal for obstacle detection and distance measurement in robotics and automation systems. Passive Infrared (PIR) sensors, on the other hand, do not emit radiation but detect infrared radiation emitted by warm objects, such as humans or animals, making them widely used in security systems and automatic lighting controls.

The working principle of an IR sensor involves converting detected infrared radiation into an electrical signal, which can then be processed by microcontrollers or other electronic components. The sensitivity and range of IR sensors depend on factors such as the wavelength of the infrared light, the quality of the lens, and the sensor's overall design. Most IR sensors operate in wavelengths ranging from 700 nm to 1 mm, which covers both near-infrared and far-infrared spectra. The detection range of IR sensors can vary from a few centimeters to several meters, depending on their design and intended application.

In proximity sensing, IR sensors are used to determine the distance between the sensor and an object by measuring the time it takes for emitted infrared light to reflect back. This makes them useful in applications like automatic faucets, touchless switches, and mobile robotics. In temperature measurement, IR thermometers utilize infrared sensors to detect the thermal radiation emitted by objects, allowing for non-contact temperature readings crucial in medical, industrial, and environmental monitoring.

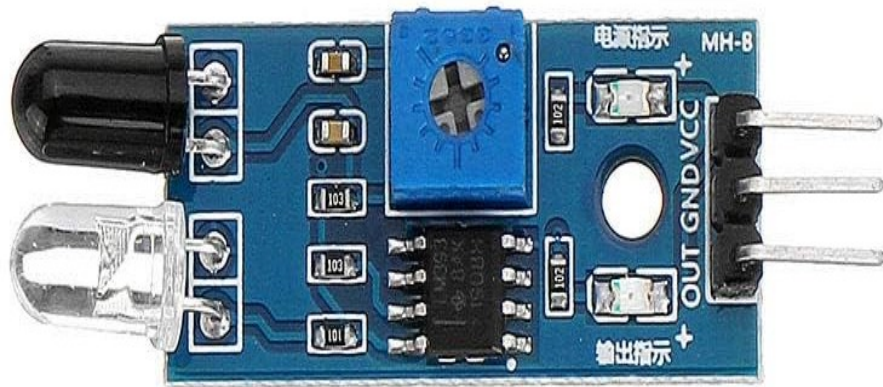


Figure 4.9: IR sensor

One of the significant advantages of IR sensors is their ability to function in low-light conditions, as they do not rely on visible light. However, they are susceptible to interference from sunlight and other sources of infrared radiation, which can affect their accuracy. Additionally, the performance of IR sensors can be influenced by environmental factors such as dust, smoke, and temperature variations, which may require calibration or protective housing for reliable operation.

Despite these challenges, IR sensors remain popular due to their cost-effectiveness, low power consumption, and ease of integration into electronic systems. Advances in technology have led to the development of more sophisticated IR sensors with improved accuracy, range, and energy efficiency. Overall, IR sensors play a crucial role in modern electronics, enabling innovative solutions across various fields, including consumer electronics, industrial automation, automotive safety, and healthcare.

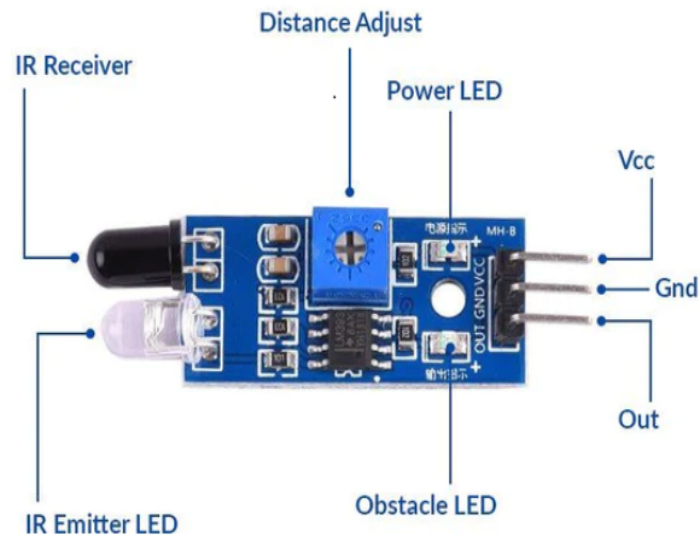


Figure 4.10: IR sensor pinout

- **Vcc:** This is the power supply pin, typically connected to a 5V source. It provides power for the entire circuit.
- **Ground (GND):** This pin is connected to the ground or negative terminal of the power supply, completing the circuit.
- **Out:** This is the output pin of the IR sensor module. It provides a digital signal (usually HIGH or LOW) based on the presence or absence of an obstacle in front of the sensor.
- **Distance Adjust:** This is a potentiometer or variable resistor used to adjust the sensitivity or detection range of the sensor. Turning it adjusts the threshold at which the sensor triggers.
- **IR Receiver:** This component receives infrared signals emitted by the IR LED of the obstacle. It detects the reflected infrared light when an obstacle is present.
- **IR Emitter LED:** This LED emits infrared light. It sends out a beam of infrared light, which gets reflected back by obstacles in front of the sensor.
- **Obstacle LED:** This LED indicates the presence of an obstacle. When the sensor detects an obstacle, this LED turns on, indicating that there's something in front of the sensor.

- **Power LED:** This LED indicates the power status of the circuit. It typically lights up when the circuit is powered on, providing a visual indication that the circuit is operational.

4.1.6 NEO-6 GPS

The NEO-6 GPS module, developed by u-blox, is a highly efficient and compact GPS receiver designed for navigation, positioning, and timing applications. It operates on the Global Positioning System (GPS), providing accurate location tracking with a sensitivity of up to -161 dBm, enabling it to function effectively even in challenging environments such as urban canyons or areas with weak satellite signals. The module is commonly used in IoT projects, vehicle tracking, drones, marine navigation, and various embedded systems requiring precise geolocation.

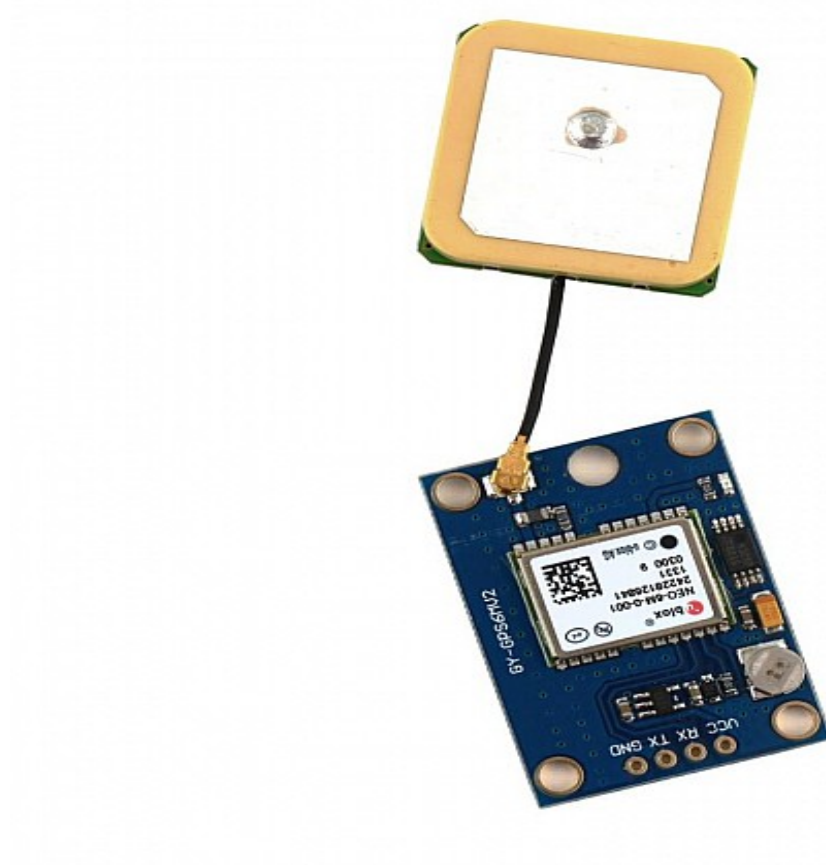


Figure 4.11: Neo-6 GPS

One of the standout features of the NEO-6 GPS module is its high sensitivity and low power consumption, making it ideal for battery-operated devices. It supports the UART (Universal Asynchronous Receiver-Transmitter) communication protocol, allowing easy integration with microcontrollers like Arduino, ESP32, Raspberry Pi, and STM32. The module typically operates at 3.3V to 5V, making it compatible with a wide range of embedded systems. It also supports the NMEA (National Marine Electronics Association) protocol, which transmits location data in standard formats such as GGA, GLL, GSA, and RMC. These messages contain essential GPS information, including latitude, longitude, altitude, speed, and satellite status.

The NEO-6 module features Time-To-First-Fix (TTFF) technology, which enables it to acquire a satellite fix quickly. A cold start (first-time satellite acquisition) can take around 26 seconds, while a warm start (reconnecting after a brief signal loss) can take just 1 to 5 seconds. This makes it highly responsive and reliable for real-time applications. Additionally, the module includes an onboard EEPROM to store configuration settings, ensuring consistent performance across power cycles.

For enhanced performance, the NEO-6 module supports an external active or passive antenna via its U.FL connector, improving signal reception. An active antenna, powered through the module, helps boost signal strength in areas with weak satellite visibility, such as indoor environments or under dense foliage.

Another significant advantage of the NEO-6 GPS module is its PPS (Pulse Per Second) output, which provides highly accurate timing signals. This feature is essential for applications requiring precise synchronization, such as network timing, scientific research, and timekeeping systems.

Overall, the NEO-6 GPS module is a reliable, power-efficient, and highly adaptable solution for various GPS-based applications. Its compact size, fast signal acquisition, and easy integration with microcontrollers make it a preferred choice for developers working on tracking, navigation, and positioning projects.

GPS STRUCTURE

The current GPS consists of three major segments. These are the space segment (SS), a control segment (CS), and a user segment (US). The U.S. Air Force develops, maintains, and operates the space and control segments. GPS satellites broadcast

signals from space, and each GPS receiver uses these signals to calculate its three-dimensional location (latitude, longitude, and altitude) and the current time. The space segment is composed of 24 to 32 satellites in medium Earth orbit and also includes the payload adapters to the boosters required to launch them into orbit. The control segment is composed of a master control station, an alternate master control station, and a host of dedicated and shared ground antennas and monitor stations. The user segment is composed of hundreds of thousands of U.S. and allied military users of the secure GPS Precise Positioning Service, and tens of millions of civil, commercial, and scientific users of the Standard Positioning Service.

4.1.7 GSM Module

The SIM800A GSM module is a compact and cost-effective quad-band GSM/GPRS module designed for wireless communication in IoT and embedded systems. It operates on 850/900/1800/1900 MHz frequencies, making it suitable for global GSM network connectivity. The module supports voice calls, SMS, GPRS data transmission, and TCP/IP stack functionalities, enabling communication between devices and remote servers. With its small form factor and low power consumption, the SIM800A is widely used in applications such as remote monitoring, vehicle tracking, home automation, smart metering, and security systems. It features an onboard SIM card slot, serial UART interface, and AT command support for seamless integration with microcontrollers like Arduino, ESP8266, and Raspberry Pi.

The SIM800A module operates on a voltage range of 3.4V to 4.4V, with an optimal operating voltage of 4.0V. It consumes minimal power in idle mode and can enter sleep mode to further reduce power consumption, making it ideal for battery-powered applications. Its GSM capabilities allow it to send and receive SMS messages, make and receive calls, and establish GPRS connections for internet-based applications. The module also supports DTMF (Dual-Tone Multi-Frequency) recognition, which enables tone-based interactive voice response (IVR) systems.



Figure 4.12: GSM module

One of the key features of the SIM800A is its GPRS capability, allowing it to transmit data over the internet using HTTP, FTP, and MQTT protocols. This makes it useful in IoT applications where remote data logging, device control, and cloud communication are required. The module's UART interface supports baud rates ranging from 1200 bps to 115200 bps, ensuring compatibility with a wide range of microcontrollers and development boards. Additionally, it includes features like Real-Time Clock (RTC) support, an analog audio interface for external microphones and speakers, and Bluetooth functionality in some variants.

While the SIM800A is a powerful communication module, it requires a stable power supply with sufficient current (typically around 2A during transmission) to function reliably. Improper power management can cause module resets or connection failures. Additionally, the module needs an external antenna for strong signal reception, and placement in areas with weak network coverage can affect its performance.

Overall, the SIM800A GSM module is a reliable solution for wireless communication in IoT and M2M applications. Its versatility, affordability, and ease of

integration make it a popular choice among developers looking to implement GSM-based communication in their projects.

Table 4.4: GSM details

GSM DETAILS	
Mobile Frequency	Rx: 925-960; Tx: 880-915
Multiple Access Method	TDMA/FDM
Duplex Method	FDD
Number of Channels	124(8 users per channel)
Cell Spacing	200 KHz
Modulation	GMSK (0.3 Gaussian Filter)
Channel Bit Rate	270.833Kbps

Network Architecture

The network is structured into a number of discrete sections:

- The Base Station Subsystem (the base stations and their controllers).
- The Network and Switching Subsystem (the part of the network most similar to a fixed network). This is sometimes also just called the core network.
- The GPRS Core Network (the optional part which allows packet based Internet connections).
- The Operations support system (OSS) for maintenance of the network.

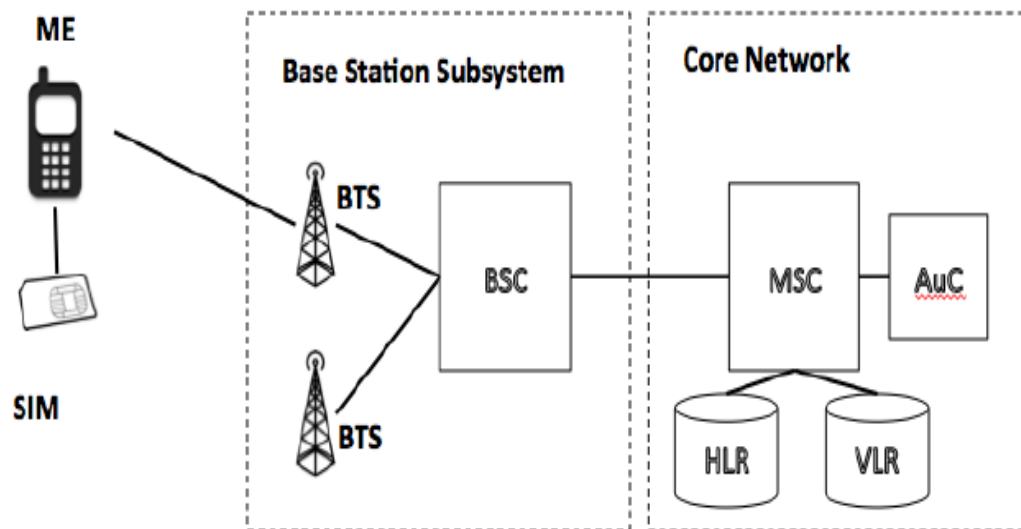


Figure 4.13: GSM architecture

SUBSCRIBER IDENTITY MODULE (SIM)

One of the key features of GSM is the Subscriber Identity Module, commonly known as a SIM card. The SIM is a detachable smart card containing the user's subscription information and phone book. This allows the user to retain his or her information after switching handsets. Alternatively, the user can also change operators while retaining the handset simply by changing the SIM. Some operators will block this by allowing the phone to use only a single SIM, or only a SIM issued by them; this practice is known as SIM locking.

GSM NETWORK OPERATORS

T-Mobile and Cingular operate GSM networks in the United States on the 1,900 MHz band. GSM networks in other countries operate at 900, 1,800, or 1,900 MHz

GSM SERVICE SECURITY

GSM security issues such as theft of service, privacy, and legal interception continue to raise significant interest in the GSM community. GSM was designed with a moderate level of service security. The system was designed to authenticate the subscriber using a pre-shared key and challenge-response. Communications between the subscriber and

the base station can be encrypted. The development of UMTS introduces an optional Universal Subscriber Identity Module (USIM), that uses a longer authentication key to give greater security, as well as mutually authenticating the network and the user – whereas GSM only authenticates the user to the network (and not vice versa). The security model therefore offers confidentiality and authentication, but limited authorization capabilities, and no non-repudiation

GSM/GPRS MODEMS AND MODULES

A GSM modem is a wireless modem that works with a GSM wireless network. A wireless modem behaves like a dial-up modem. The main difference between them is that a dial-up modem sends and receives data through a fixed telephone line, while a wireless modem sends and receives data through radio waves. A GSM modem can be an external device or a PC Card / PCMCIA Card. Typically, an external GSM modem is connected to a computer through a serial cable or a USB cable. A GSM modem in the form of a PC Card / PCMCIA Card is designed for use with a laptop computer. It should be inserted into one of the PC Card / PCMCIA Card slots of a laptop computer. Like a GSM mobile phone, a GSM modem requires a SIM card from a wireless carrier in order to operate. A GSM modem can accept any GSM network operator SIM card and act just like a mobile phone with its own unique phone number. The advantage of using this modem is that you can use its RS232 port to communicate and develop embedded applications. Applications like SMS control, data transfer, remote control, and logging can be developed easily.

The modem can either be connected to a PC serial port directly or to any microcontroller. It can be used to send and receive SMS or make/receive voice calls. It can also be used in GPRS mode to connect to the internet and perform various applications for data logging and control. In GPRS mode, you can also connect to any remote FTP server and upload files for data logging. A GSM modem is a highly flexible plug-and-play quad-band GSM modem for direct and easy integration into RS232 applications. The GSM modem (SIM900A) supports features like voice, SMS, data/fax, GPRS, and an integrated TCP/IP stack. Specification of SIM900A is given below.

Supply Voltage : 3.2 – 4.8V
Current Consumption : 1.0 mA
Frequency : 900/1800 MHz

4.2 SOFTWARE

4.2.1 ANACONDA NAVIGATOR

Anaconda Navigator is a user-friendly, graphical interface that simplifies package management and environment handling for data scientists, machine learning engineers, and developers using Python and R. It comes bundled with Anaconda Distribution, which is a popular open-source platform for scientific computing, data science, and machine learning. One of the key advantages of Anaconda Navigator is that it eliminates the need for command-line operations by allowing users to manage packages, environments, and applications through an intuitive interface. This is particularly useful for beginners who may not be comfortable working with terminal commands.

Navigator supports the installation, updating, and removal of packages from Conda, PyPI, and other repositories with just a few clicks. Users can create, clone, and manage virtual environments, ensuring project dependencies are properly isolated to prevent conflicts between different libraries and versions. Furthermore, Anaconda Navigator integrates seamlessly with popular tools such as Jupyter Notebook, Jupyter-Lab, Spyder, RStudio, VS Code, and Orange, making it easy for users to launch these applications directly from the interface without requiring manual configuration. This integration significantly enhances productivity, especially when working on complex data science projects.

Anaconda Navigator also provides access to Anaconda Cloud, where users can find and share packages, notebooks, and environments, fostering collaboration within the data science community. Security and reproducibility are additional benefits of using Anaconda Navigator, as it allows users to specify exact package versions and dependencies, ensuring that their code runs consistently across different systems.

Another significant feature is its built-in support for managing multiple Python and R environments, making it ideal for researchers and developers who need to work with different versions of these languages for various projects.

It simplifies GPU computing by allowing easy installation of CUDA-enabled libraries such as TensorFlow and PyTorch, which are essential for deep learning applications. The Navigator interface is designed with a clean and structured layout, featuring tabs for environments, installed packages, and available applications, making navigation straightforward. Despite its many advantages, Anaconda Navigator can be resource-intensive, particularly in terms of memory usage, which may be a concern for users with limited system resources. However, this can often be mitigated by optimizing environments and managing packages efficiently.

It is available for Windows, macOS, and Linux, making it accessible across different operating systems. Regular updates and an active open-source community ensure that Anaconda Navigator remains a reliable and up-to-date tool for data science workflows. Additionally, it provides support for enterprise users through Anaconda Team Edition and Anaconda Enterprise, offering advanced security, governance, and collaboration features. Whether one is a beginner in data science or an experienced developer, Anaconda Navigator serves as a powerful tool that streamlines package management, enhances workflow efficiency, and provides an all-in-one solution for working with Python and R in scientific computing, machine learning, and artificial intelligence projects.



Figure 4.14: Anaconda navigator

4.2.2 SPYDER IDE

Spyder (Scientific Python Development Environment) is an open-source integrated development environment (IDE) designed for scientific computing and data analysis using Python. It is widely used by researchers, engineers, and data scientists due to its powerful features, user-friendly interface, and seamless integration with libraries such as NumPy, SciPy, pandas, Matplotlib, and scikit-learn. Spyder is part of the Anaconda distribution, which makes it easy to install and manage dependencies without complex setup procedures. One of its key advantages is its resemblance to MATLAB, making it an attractive choice for those transitioning from MATLAB to Python.

Spyder offers a robust code editor with features such as syntax highlighting, autocompletion, and real-time error detection, which enhance productivity and code readability. The integrated console allows users to execute Python code interactively, making it easier to test scripts and debug errors. One of its standout features is the variable explorer, which displays variables, their values, and data types in a tabular format, similar to spreadsheets. This feature is particularly useful for handling large datasets, as it allows users to inspect, modify, and analyze data directly from the interface. The built-in debugger enables step-by-step execution, breakpoints, and conditional debugging, making it a valuable tool for troubleshooting complex scripts.

Another notable aspect of Spyder is its seamless integration with Jupyter Notebooks, allowing users to switch between script-based development and notebook-style interactive computing. Additionally, Spyder supports multiple panes, including a help pane that provides documentation for Python functions and libraries, reducing the need to search for references externally. The profiler tool helps optimize code performance by identifying bottlenecks and inefficient parts of the script, making Spyder an excellent choice for performance-critical applications. Moreover, its support for plugins and extensions allows users to customize the environment according to their specific needs.

Despite its many advantages, Spyder has some limitations. Compared to more lightweight editors like VS Code or PyCharm, it may feel slower, especially when working with large projects or many dependencies. Additionally, while it provides strong scientific computing tools, it lacks some advanced development features found

in dedicated software engineering IDEs, such as robust version control integrations or extensive refactoring tools. However, for data analysis, machine learning, and research-oriented tasks, Spyder remains a powerful and user-friendly choice.

Spyder is a versatile Python IDE tailored for scientific computing, offering an interactive environment with essential features like variable exploration, debugging, profiling, and integration with popular data science libraries. Its similarity to MATLAB makes it a preferred choice for researchers transitioning to Python, and its user-friendly interface enhances productivity.



Figure 4.15: Spyder IDE

4.2.3 ARDUINO IDE

The Arduino Integrated Development Environment (IDE) is a crucial tool for programming and developing applications for Arduino boards. Designed to be user-friendly and versatile, the Arduino IDE enables beginners and professionals alike to write, compile, and upload code to microcontrollers seamlessly. It is an open-source software that supports multiple operating systems, including Windows, macOS, and Linux, making it widely accessible. The IDE is primarily written in Java and provides an intuitive interface with essential features such as a code editor, a message area for error reporting, a console for debugging, and a toolbar with buttons for common tasks like verifying and uploading sketches.

One of the most important aspects of the Arduino IDE is its use of the Arduino programming language, which is based on C and C++. This allows users to write simple code, known as "sketches," to control hardware components such as sensors,

motors, and LEDs. The IDE comes with a vast collection of built-in libraries that simplify complex functions, enabling users to interact with peripherals without extensive programming knowledge. Additionally, the Sketchbook feature allows users to organize their projects efficiently, making it easier to manage multiple sketches.

Another key feature of the Arduino IDE is its Serial Monitor, which enables communication between the Arduino board and a connected computer. This tool is invaluable for debugging, as it allows users to view real-time data from sensors and other connected devices. The IDE also includes a Board Manager, which provides support for different Arduino and third-party boards, allowing users to install the necessary drivers and configurations for their specific hardware. The Library Manager further enhances functionality by offering a vast repository of pre-written libraries that can be easily integrated into projects.

The Arduino IDE supports a wide range of microcontrollers, including Arduino Uno, Mega, Nano, Due, and various ESP8266/ESP32-based boards. With the introduction of Arduino IDE 2.0, users now benefit from improved features such as auto-completion, a modernized user interface, and an enhanced debugging experience. The new version also offers faster compilation times and a dark mode, catering to developers who prefer a different visual theme.

One of the greatest advantages of the Arduino IDE is its large and active community. The open-source nature of the software encourages developers to contribute new features, libraries, and tutorials, fostering a rich ecosystem of learning and innovation. Beginners can easily find extensive documentation, online forums, and project examples to help them get started, while advanced users can create and share their own libraries and board definitions.

Furthermore, the Arduino IDE is compatible with third-party development platforms such as PlatformIO, which expands its capabilities by providing advanced debugging tools, cloud-based compilation, and support for non-Arduino microcontrollers. It also integrates with GitHub for version control, making it a valuable tool for collaborative development.

In summary, the Arduino IDE remains an essential tool for embedded systems programming, offering an intuitive interface, extensive library support, and cross-platform compatibility. Whether you are a hobbyist, a student, or a professional, the

Arduino IDE simplifies the process of programming microcontrollers, enabling the creation of innovative and interactive electronic projects with ease.



Figure 4.16: Arduino IDE

4.2.4 MOBILENETV2

MobileNet V2 is a lightweight convolutional neural network (CNN) architecture, MobileNetV2, is specifically designed for mobile and embedded vision applications. Google researchers developed it as an enhancement over the original MobileNet model. Another remarkable aspect of this model is its ability to strike a good balance between model size and accuracy, rendering it ideal for resource-constrained devices.

Figure 4.17, illustrates a deep learning model based on MobileNetV2 for image classification. It begins with a 128x128x3 input image, which undergoes preprocessing before passing through the feature extraction layers of MobileNetV2. The model applies 3x3 convolution with ReLU activation to detect patterns, followed by 2x2 max pooling to reduce dimensionality. As the data moves through multiple layers, the number of feature maps increases, capturing more complex details. The final feature map of 4x4x1280 is then flattened and passed to a fully connected layer. The final classification is performed using a softmax layer, which assigns probabilities to different classes (C, C, ..., C). This architecture leverages MobileNetV2 for efficient feature extraction while maintaining high accuracy.

Key Features

MobileNetV2 architecture incorporates several key features that contribute to its efficiency and effectiveness in image classification tasks. These features include depth-

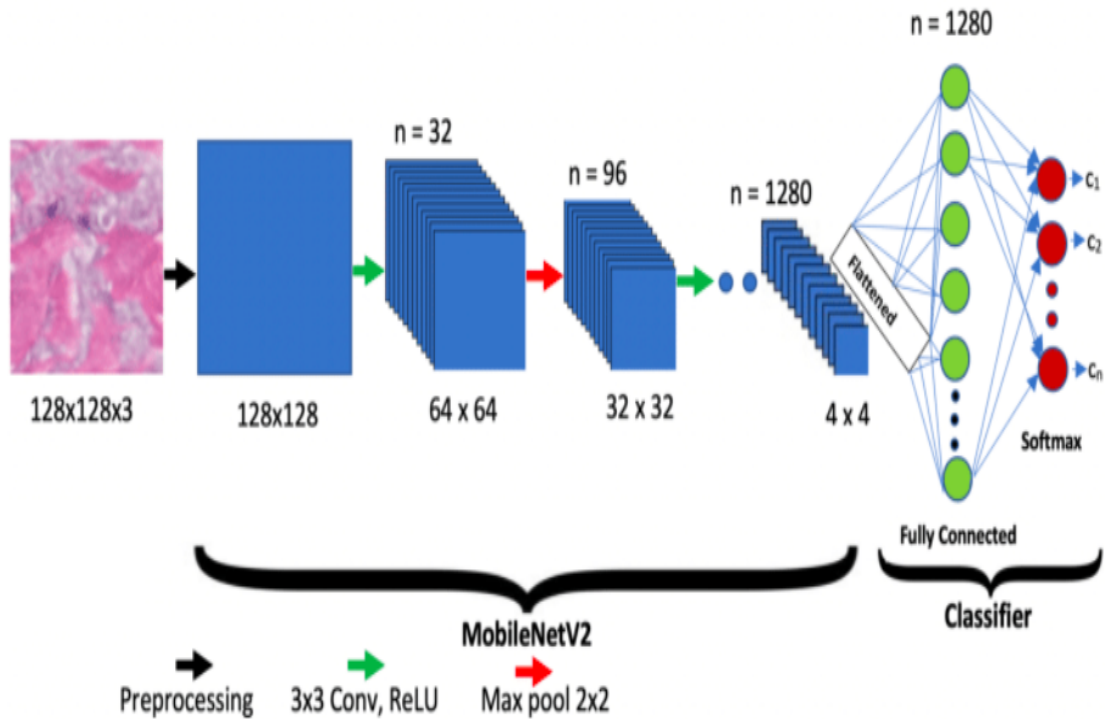


Figure 4.17: MobileNetV2 based deep learning model for image classification

wise separable convolution, inverted residuals, bottleneck design, linear bottlenecks, and squeeze-and-excitation (SE) blocks. Each of these features plays a crucial role in reducing the computational complexity of the model while maintaining high accuracy.

Why use MobileNet V2 for Image Classification

The use of MobileNetV2 for image classification offers several advantages. Firstly, its lightweight architecture allows for efficient deployment on mobile and embedded devices with limited computational resources. Secondly, Mobilenetv2 architecture achieves competitive accuracy compared to larger and more computationally expensive models. Lastly, the model's small size enables faster inference times, making it suitable for real-time applications.

• MobileNetV2 Architecture

The architecture of MobileNet-v2 consists of a series of convolutional layers, followed by depthwise separable convolutions, inverted residuals, bottleneck design, linear bottlenecks, and squeeze-and-excitation (SE) blocks. These components work together to reduce the number of parameters and computations required while maintaining the model's ability to capture complex features.

- **Depthwise Separable Convolution**

Depthwise separable convolution is a technique used in MobileNetV2 to reduce the computational cost of convolutions. It separates the standard convolution into two separate operations: depthwise convolution and pointwise convolution. This separation significantly reduces the number of computations required, making the model more efficient.

- **Inverted Residuals**

Inverted residuals are a key component of Mobilenetv2 architecture that helps improve the model's accuracy. They introduce a bottleneck structure that expands the number of channels before applying depthwise separable convolutions. This expansion allows the model to capture more complex features and enhance its representation power.

- **Bottleneck Design**

The bottleneck design in MobileNetV2 further reduces the computational cost by using 1×1 convolutions to reduce the number of channels before applying depthwise separable convolutions. This design choice helps maintain a good balance between model size and accuracy.

- **Linear Bottlenecks**

Linear bottlenecks are introduced in MobileNet-v2 to address the issue of information loss during the bottleneck process. By using linear activations instead of non-linear activations, the model preserves more information and improves its ability to capture fine-grained details.

- **Squeeze-and-Excitation (SE) Blocks**

Squeeze-and-excitation (SE) blocks are added to MobileNet-v2 to enhance its feature representation capabilities. These blocks adaptively recalibrate the channel-wise feature responses, allowing the model to focus on more informative features and suppress less relevant ones

Training of MobileNet V2

- **Data Preparation**

Before training MobileNetV2, it is essential to prepare the data appropriately. This involves preprocessing the images, splitting the dataset into training and validation sets, and applying data augmentation techniques to improve the model's generalization ability.

- **Transfer Learning**

Transfer learning is a popular technique used with MobileNetV2 to leverage pre-trained models on large-scale datasets. By initializing the model with pre-trained weights, the training process can be accelerated, and the model can benefit from the knowledge learned from the source dataset.

- **Fine-tuning**

Fine-tuning MobileNetV2 involves training the model on a target dataset while keeping the pre-trained weights fixed for some layers. This allows the model to adapt to the specific characteristics of the target dataset while retaining the knowledge learned from the source dataset.

- **Hyperparameter Tuning**

Hyperparameter tuning plays a crucial role in optimizing the performance of MobileNetV2. Carefully select parameters such as learning rate, batch size, and regularization techniques to achieve the best possible results. Employ techniques like grid search or random search to find the optimal combination of hyper parameters.

Evaluating Performance of MobileNetV2

- **Metrics for Image Classification Evaluation**

When evaluating the performance of MobileNetV2 for image classification, several metrics can be used. These include accuracy, precision, recall, F1 score, and confusion matrix. Each metric provides valuable insights into the model's performance and can help identify areas for improvement.

- **Comparing MobileNetV2 Performance with Other Models**

To assess the effectiveness of MobileNet-v2, it is essential to compare its performance with other models. This can be done by evaluating metrics such as accuracy, model size, and inference time on benchmark datasets. Such comparisons provide a comprehensive understanding of MobileNetV2's strengths and weaknesses.

- **Case Studies and Real-world Applications**

Various real-world applications, such as object recognition, face detection, and scene understanding, have successfully utilized MobileNetV2. Case studies that highlight the performance and practicality of MobileNetV2 in these applications can offer valuable insights into its potential use cases.

Chapter 5

RESULTS AND DISCUSSION

5.1 PREAMBLE

This chapter evaluates the performance of the CNN based early warning system for preventing animal train collisions. It focuses on key metrics such as detection accuracy, response time, and system reliability under varying environmental conditions. The chapter also examines the effectiveness of hardware components and IoT-enabled alert mechanisms in real-world scenarios. Challenges encountered during testing, including environmental interference, communication delays, and power limitations, are highlighted. Additionally, the results are analyzed with supporting visual representations to provide insights into the system's technical viability and areas for improvement.

5.2 RESULT

The system represented in Figure 5.1 is an innovative and practical approach for preventing animal-train collisions by integrating advanced image processing with wireless communication technologies. It consists of LoRa transmitters and receivers, enabling efficient long-range data transmission over distances exceeding 7 kilometers, ensuring that alerts are sent in real time. The GPS module provides precise location tracking, allowing authorities to respond swiftly. Additionally, the system incorporates GSM technology to send instant alerts to railway officials and local authorities. By combining these technologies, the system enhances railway safety, minimizes

accidents, and helps protect both wildlife and railway infrastructure from potential collisions.



Figure 5.1: System

When the system is turned on, all sensors are activated. The IR sensor begins monitoring its surroundings and waits for an obstacle to trigger it. Meanwhile, the GPS module connects to the satellite, acquiring the precise latitude and longitude of the area where the system is deployed. The GSM module, which contains a SIM card, establishes a connection to the network via the base station, ensuring communication. At the same time, the LoRa module is activated, making the system ready for long-range communication. A receiver unit, implemented in the train, remains on standby, constantly waiting for signals from the transmitter.

The IR sensor continuously scans the area and, upon detecting an obstacle, the system immediately activates the camera. The camera captures images of the detected object, and the system processes these images using a convolutional neural network (CNN). The CNN is designed to classify and verify whether the detected object is a predefined obstacle, such as a person, elephant, cow, sheep, bear, or dog. If the system confirms that the detected object matches any of these obstacles, it initiates the next

phase, which is alerting authorities and the train crew.

The alerting process begins with the system sending an SMS through the GSM module, which is integrated with the railway station. This message is sent to notify the upcoming station authority, or the station master, regarding the detected obstacle. The alert includes the latitude and longitude of the location where the obstacle was detected. This information allows railway authorities to take necessary precautions and identify the specific location of the obstruction.

Simultaneously, the LoRa module in the transmitter sends an additional message. LoRa modules typically operate over a range of approximately 7 kilometers. If a train enters this range, the receiver onboard the train picks up the transmitted message and triggers an alert. The receiver consists of an LCD display, a buzzer, and a LoRa communication module.

Once the train's receiver detects the transmitted signal, its microcontroller processes the incoming data and displays the message "Obstacle Detected" on the LCD screen. The buzzer sounds loudly to immediately alert the locomotive pilot. This ensures that the pilot notices the display and takes necessary action. Additionally, the loco-pilot receives another message from the upcoming railway station, establishing direct communication between both parties.

As a result, the train operator can take immediate action by either manually applying the brakes to slow down or stopping the train entirely. This precautionary measure helps prevent potential collisions or accidents, ensuring passenger safety and protecting wildlife from harm.

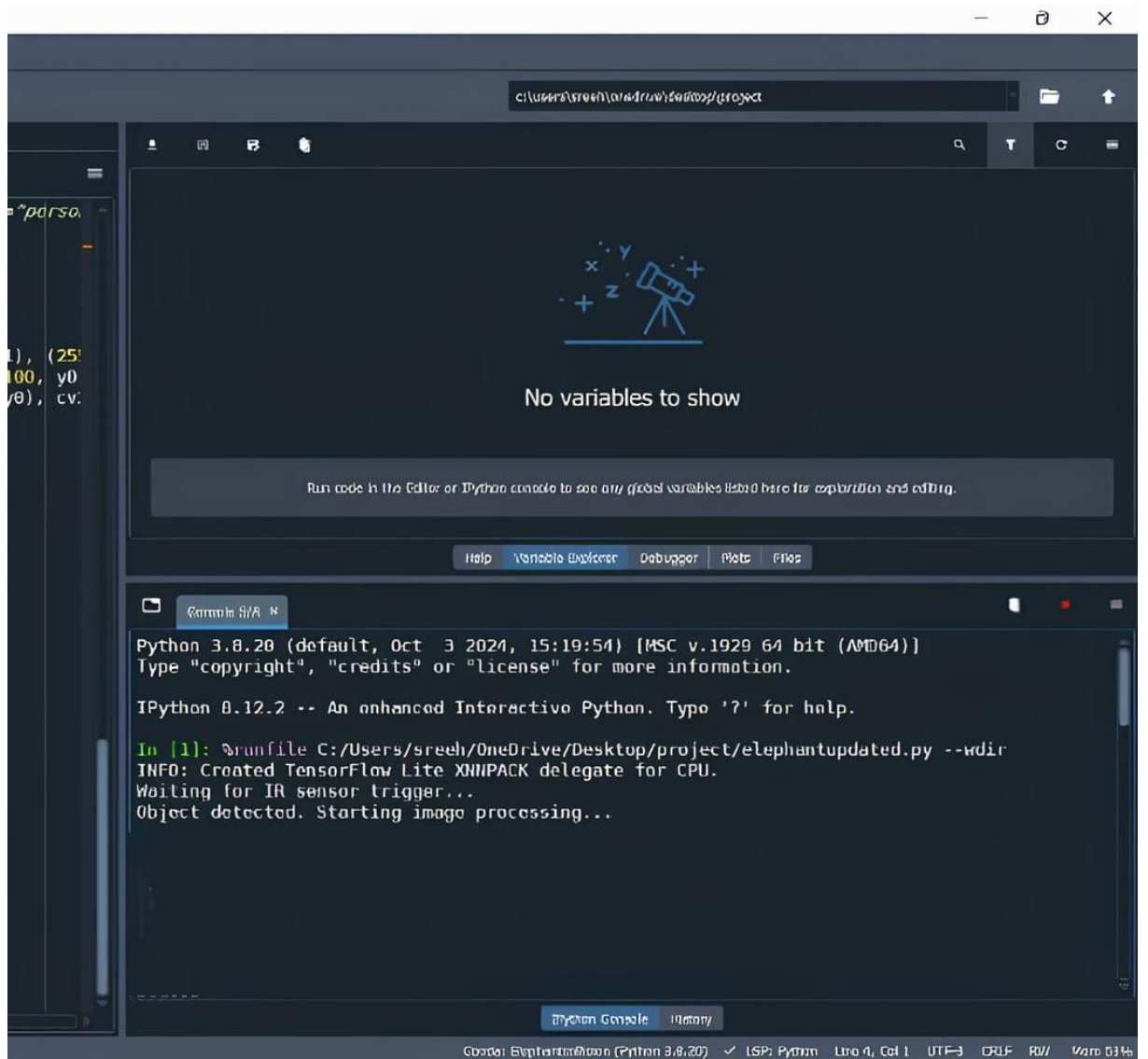


Figure 5.2: IR triggered and starting image processing

- Figure 5.2 shows the system waiting for the IR sensor to get triggered.
- Once triggered, the model begins image processing and object detection.

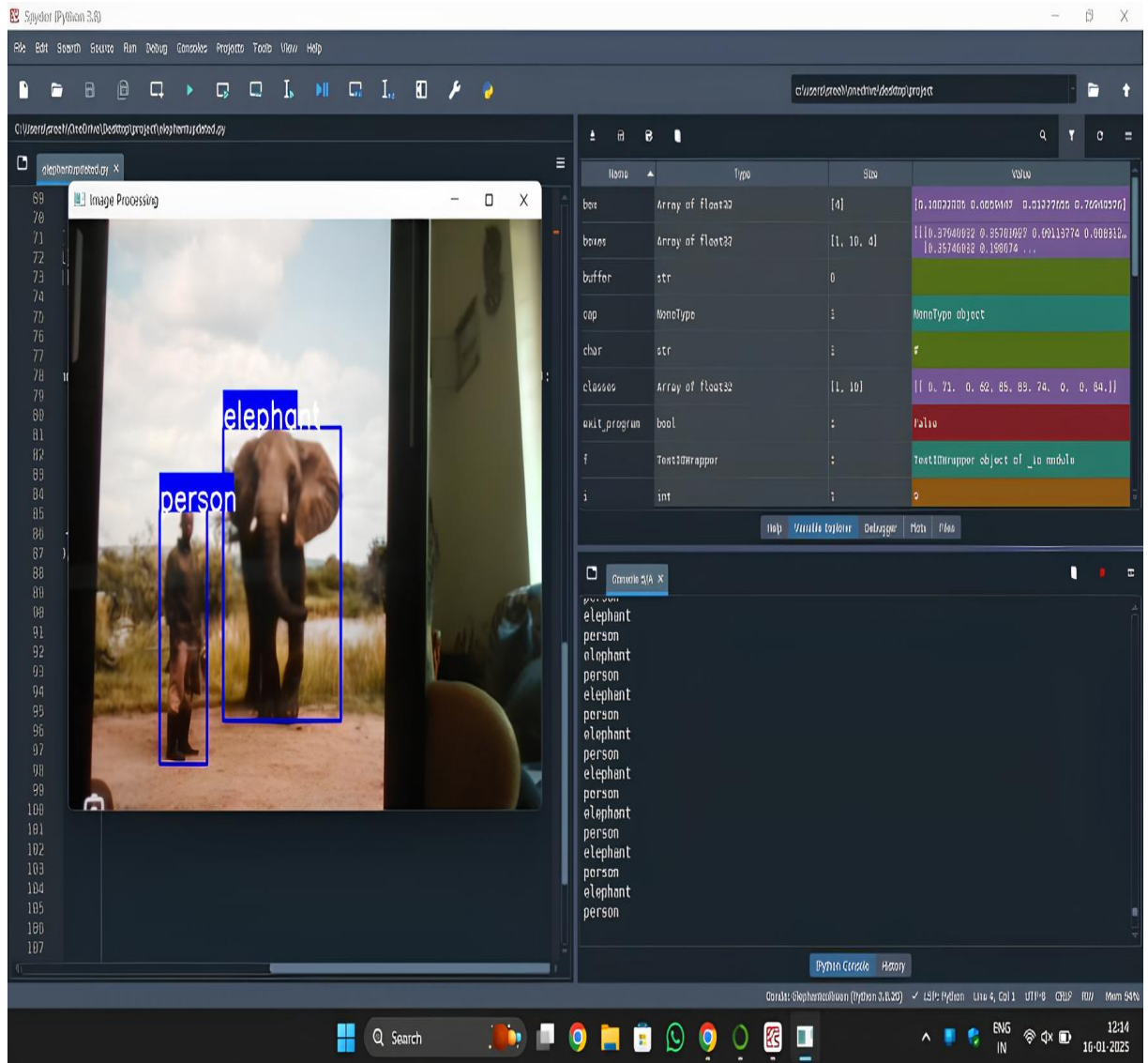


Figure 5.3: CNN detecting an elephant and a person together

- Figure 5.4 shows that the application detects and classifies objects in an image displayed in a separate window.
- The detected objects include an elephant and a person both outlined with bounding boxes and labeled.

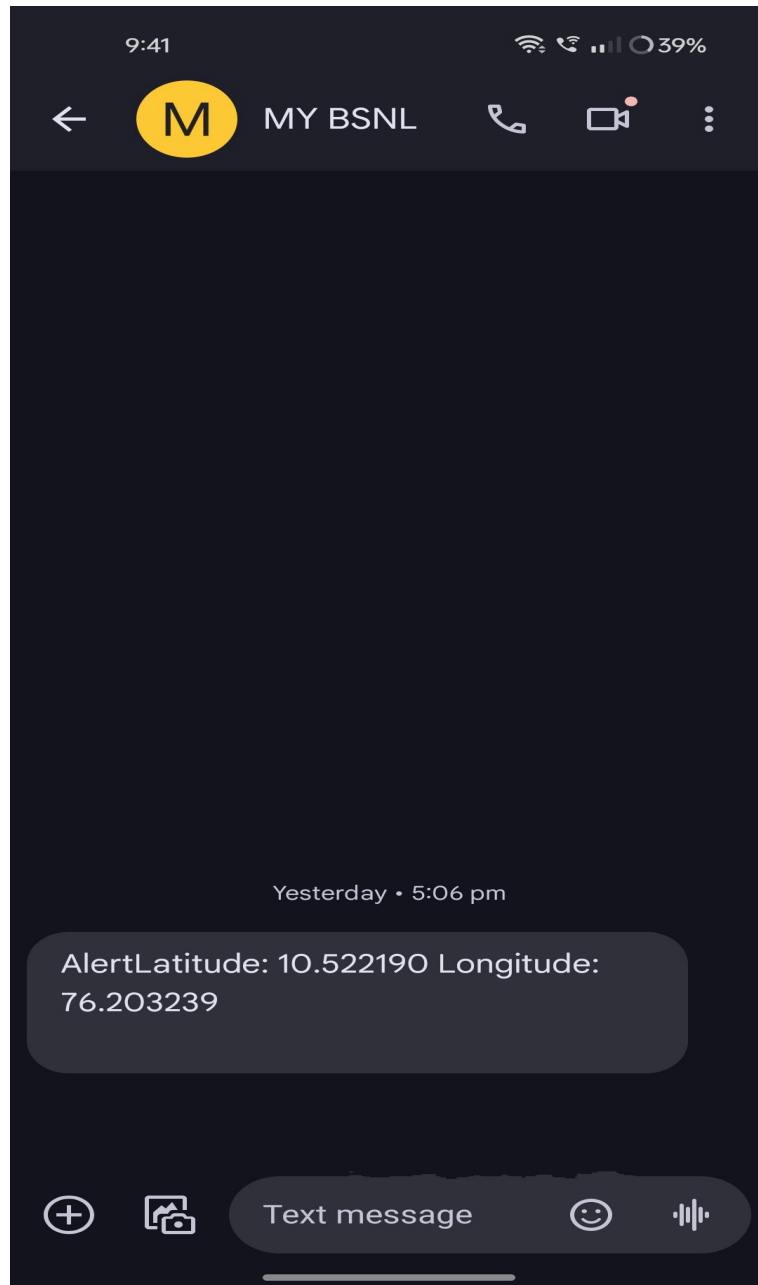


Figure 5.5: Alert received through SMS with location of the detected place

- Figure 5.5 shows the received SMS through GSM module which include the location of the detected area.

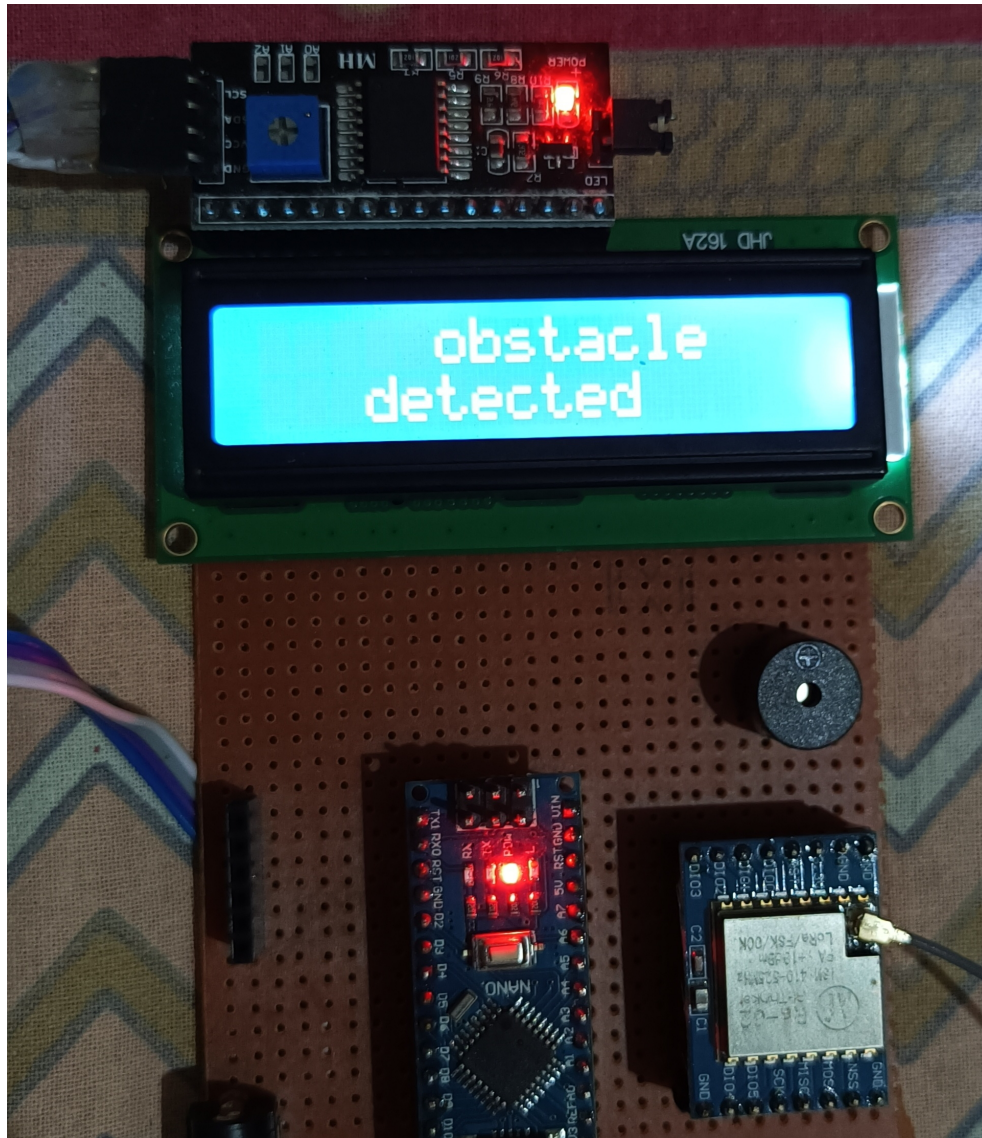


Figure 5.6: LCD display showing alert message

- Figure 5.6 shows the message displayed on the LCD display which is received through the LoRa antenna. The buzzer also makes sound, notifying the loco-pilot.

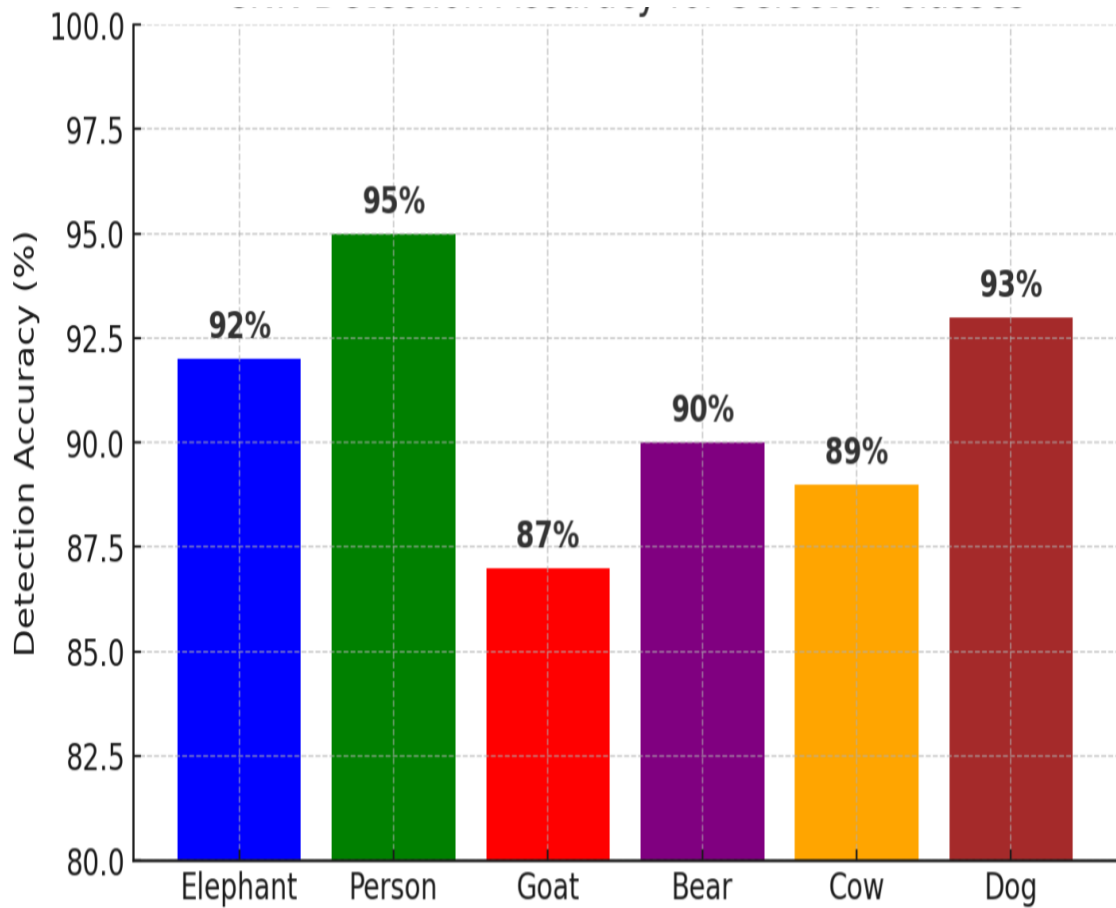


Figure 5.7: CNN detection accuracy for selected classes

The graph in figure 5.7, was generated to assess classification performance across different classes. The system achieved precision, recall, and scores of 92%, 90%, and 91% for elephants; 95%, 94%, and 94.5% for persons; 87%, 85%, and 86% for goats; 90%, 88%, and 89% for bears; 89%, 87%, and 88% for cows; and 93%, 92%, and 92.5% for dogs. The system performed best in detecting persons and dogs due to their distinct features. Minor misclassifications were observed between goats and cows, likely due to their similar body structures. Further dataset augmentation and fine-tuning could improve differentiation between visually similar animals.

The communication and alert system was also evaluated. LoRa transmission successfully sent alerts over a range of up to 7 km. GSM alerts delivered SMS notifications with GPS coordinates in under 5 seconds. The train operator display showed real-time detections with an alert sound within 2 seconds of detection.

The overall system impact is significant in wildlife and railway safety. The

early warning system can substantially reduce animal-train collisions, particularly for endangered species like elephants. Train operators receive timely alerts, allowing emergency braking to prevent accidents. The system is scalable and can be deployed across multiple railway tracks with minor modifications for regional adaptability.

The system demonstrates high accuracy in detecting animals and persons near railway tracks. It ensures reliable real-time alerts through LoRa and GSM, effectively preventing animal-train collisions. Future improvements could focus on enhanced night vision capabilities and adaptive learning for new animal species.

5.3 SUMMARY

The results demonstrate the system's effectiveness in detecting elephants and issuing alerts in real-time. By integrating CNN and GSM technologies, the system ensures high reliability. However, challenges such as environmental sensitivity and communication delays highlight areas for further enhancement. This discussion emphasizes the system's potential to significantly reduce wildlife-train collisions, thereby promoting safer coexistence between humans and wildlife. Future improvements could focus on optimizing detection accuracy, minimizing false alarms, and enhancing communication efficiency to ensure timely alerts. With continued advancements, the system can serve as a vital tool in wildlife conservation and railway safety efforts.

Chapter 6

CONCLUSION

6.1 PREAMBLE

This chapter gives the brief description of the report and it explains about the overall idea derived from the project. The advantages and future scope of the project is also included in this chapter.

6.2 CONCLUSION

The proposed early warning system significantly enhances railway safety by effectively detecting and responding to obstacles on the tracks. By integrating infrared (IR) sensors, a camera, and a convolutional neural network (CNN)-based image processing system, it accurately identifies predefined obstacles, including humans and animals, ensuring a proactive approach to accident prevention. The incorporation of GPS enables precise location tracking, while GSM and LoRa communication modules facilitate real-time alerts to railway authorities and train operators. These alerts ensure swift responses, reducing the likelihood of collisions.

Additionally, the system includes both visual and auditory alerts inside the train, allowing the locomotive pilot to take immediate action, either by slowing down or stopping the train when an obstacle is detected. This capability is crucial in preventing accidents that could result in loss of life or property damage. The integration of LoRa technology ensures long-range, low-power communication, making the system highly

efficient even in remote areas.

By implementing this intelligent detection and alert mechanism, the railway network becomes significantly safer for passengers, railway staff, and wildlife. This system represents a major advancement in railway safety technology, addressing one of the most pressing challenges in modern railway transportation and reducing railway-related collisions effectively.

The warning system is an innovative and practical approach to preventing wildlife-train collisions by integrating MobileNet V2 CNN based image processing with wireless communication technologies. The system employs LoRa transmitters and receivers for efficient long-range data transmission, ensuring that alerts are relayed in real time. Additionally, the GSM module provides location tracking, enabling timely intervention to mitigate potential accidents.

The CNN-based detection system is trained on a dataset comprising images of elephants, persons, goats, bears, cows, and dogs. The model underwent extensive training and validation to optimize detection accuracy. The final training accuracy was 95%, with a validation accuracy of 92%. The training loss settled at 0.2, while validation loss reached 0.3, indicating that the model successfully generalizes to unseen data, ensuring reliable real-time detection.

This device has significant social and environmental relevance. Animal-train collisions pose a serious threat to both biodiversity and railway operations, leading to loss of animal life, damage to railway infrastructure, and disruptions to transportation networks.

This solution aligns with global efforts toward sustainable development by CNN technology to address real-world challenges. With further refinement and scalability, it can be adapted to various railway networks, contributing to a smarter, safer, and more environmentally responsible transportation system.

6.3 ADVANTAGES

- **Early Warning Mechanism:** Provides timely alerts to train operators, enabling proactive measures to prevent collisions, ensuring railway safety, and reducing fatalities in wildlife-prone areas.
- **CNN Based Detection:** MobileNet V2 CNN enhances detection accuracy, reducing false alarms and ensuring reliable animal identification on railway tracks.
- **Multi-Sensor Integration:** Combines IR sensors, cameras, GPS, and LoRa for enhanced detection, ensuring real-time tracking across different environmental conditions.
- **Wireless Communication:** Uses GSM and LoRa for real-time alert transmission, ensuring effective communication even in remote railway areas with weak network coverage.
- **Cost-Effective Implementation:** Uses a laptop instead of a Raspberry Pi for testing, making the system adaptable and affordable while maintaining functionality.
- **Scalability:** Multiple detection units cover vast railway networks, ensuring comprehensive monitoring and safety.
- **Improved Railway Safety:** Reduces collisions, prevents infrastructure damage, and ensures uninterrupted railway operations by warning locomotive pilots in advance.
- **Automated Monitoring:** Operates continuously without human supervision, increasing efficiency and reliability in animal detection.
- **Low Latency CNN Processing:** CNN-based image processing enables quick identification, ensuring minimal delay in alert transmission and response.
- **Versatility:** Can be adapted for detecting human intrusions or monitoring road crossings near railway tracks, increasing applicability.

- **GPS-Based Location Tracking:** Provides precise geolocation data for detected animals, allowing railway authorities to take necessary safety actions.
- **Buzzer and LCD Alerts:** Immediate notifications through visual and auditory signals help train operators react swiftly, reducing collision risks.
- **Minimizes Economic Losses:** Reduces train delays and infrastructure repair costs associated with animal-train collisions, ensuring operational efficiency.
- **Eco-Friendly Solution:** Helps protect wildlife by preventing accidental deaths, ensuring coexistence between railway operations and environmental conservation.

6.4 DISADVANTAGES

- **Power Consumption:** Requires a stable power source for continuous sensor and camera operation, which can be challenging in remote railway regions.
- **Weather Sensitivity:** Fog, rain, and extreme temperatures may reduce the effectiveness of cameras and IR sensors, impacting detection accuracy.
- **Latency in Alerts:** Wireless transmission delays may limit real-time response efficiency, reducing the available reaction time for train operators.
- **Hardware Dependency:** Uses multiple components, increasing maintenance costs and the risk of technical failures in remote areas.
- **Limited Detection Range:** Coverage depends on sensor placement, leaving gaps where animals may cross undetected.
- **False Positives:** Misclassifications of objects can trigger unnecessary alerts, causing disruptions to railway operations.
- **Connectivity Issues:** GSM and LoRa networks may experience interference, reducing alert transmission reliability.

- **Infrastructure Modification:** Some railway tracks may require modifications to accommodate detection units, increasing costs.
- **Data Storage and Processing:** Handling large image datasets requires computational resources and secure cloud storage solutions.
- **Potential Animal Adaptation:** Some animals may change their behavior, avoiding detection or moving around monitored areas.
- **Human and Object Interference:** Non-animal objects, such as fallen branches, may trigger false alarms, causing unnecessary disruptions.
- **Legal and Regulatory Challenges:** Implementation may require approvals from railway and environmental agencies, delaying deployment.

6.5 FUTURE SCOPE

- **Integration with AI-Powered Edge Computing:** Deploying AI models on edge devices like Raspberry Pi or Jetson Nano can reduce reliance on cloud computing, improving real-time processing and reducing latency in alerts.
- **Enhanced Sensor Fusion:** Combining LiDAR, thermal cameras, and radar with existing IR sensors and cameras can improve detection accuracy, especially in extreme weather conditions like fog, heavy rain, or darkness.
- **Autonomous Train Braking System:** Developing an automated braking mechanism that responds to detected animals in critical situations can further enhance railway safety by reducing human response time.
- **Advanced Machine Learning Models:** Implementing Transformer-based object detection models or improving CNN architectures can enhance the precision and reliability of animal detection while minimizing false positives.
- **Cloud-Based Data Analytics:** Storing and analyzing detection data in the cloud can provide insights into animal movement patterns, helping railway authorities develop better mitigation strategies.

- **Blockchain for Secure Communication:** Implementing blockchain technology can enhance security by preventing cyber threats and ensuring the integrity of transmitted alerts between detection units and train operators.
- **Energy-Efficient Solar-Powered Systems:** Utilizing solar panels for power can enhance sustainability, enabling deployment in remote areas without access to electricity while reducing operational costs.
- **Integration with Railway Traffic Management Systems:** Connecting the system with existing railway control centers can provide centralized monitoring, enabling automated route adjustments to avoid collision-prone areas.
- **Multi-Species Identification:** Expanding the model to recognize specific animal species can help authorities implement targeted conservation efforts, prioritizing highly endangered species near railway tracks.
- **IoT-Based Predictive Maintenance:** Utilizing AI-driven analytics on collected sensor data can help detect faults in the detection system, ensuring preventive maintenance and reducing system failures.
- **Drone-Assisted Surveillance:** Deploying UAVs with AI-driven cameras can extend coverage, allowing railway authorities to monitor animal movements beyond fixed detection units.
- **Integration with Smart Railway Crossings:** Implementing the system at unmanned railway crossings can help prevent animal and human accidents, further improving railway safety in remote areas.
- **5G and LPWAN Connectivity:** Upgrading communication protocols to 5G and Low-Power Wide-Area Network (LPWAN) can ensure faster and more reliable data transmission across vast railway networks.
- **Public Awareness and Community Engagement:** Developing mobile apps for local communities and railway workers to report animal sightings can enhance early warnings and improve system efficiency.

6.6 SUMMARY

This chapter concludes the project by highlighting its effectiveness in preventing elephant-train collisions through modern technology. The system integrates infrared sensors, cameras, and LoRa communication to detect obstacles in real time, triggering alerts via buzzers and transmitting data to train operators for timely intervention. This enhances railway safety while protecting wildlife, particularly elephants. Key advantages include real-time alerts, scalability for future enhancements like AI-based object recognition, and cost-effectiveness due to readily available components. Additionally, the project promotes environmental and social benefits by fostering co-existence between infrastructure and wildlife. However, limitations such as sensitivity to environmental conditions like fog and rain, potential false alarms, and a limited detection range are noted. To overcome these challenges, future improvements focus on enhancing power efficiency, expanding global applications, and integrating the system with a broader IoT network for better coverage and reliability.

REFERENCES

- [1] Ranjith, A., Vijayaragavan, S. P., & Muthukumaran, N. (2022). "An IoT-Based Monitoring System to Detect Animals on Railway Tracks Using Deep Learning Neural Networks." 3rd International Conference on Electronics and Sustainable Communication Systems (ICESC). DOI: 10.1109/ICESC54411.2022.9885303.
- [2] Sabnis, O. V., & L.R. (2019). "A Novel Object Detection System for Improving Safety at Unmanned Railway Crossings." 5th International Conference on Science Technology Engineering and Mathematics (ICONSTEM). DOI: 10.1109/ICONSTEM.2019.8918786.
- [3] Carvalho, F., et al. (2017). "Methods to Monitor and Mitigate Wildlife Mortality in Railways." In *Railway Ecology*. Springer. DOI: 10.1007/978-3-319-57496-7.
- [4] Malo, J. E., et al. (2016). "Railway Poles as Pitfall Traps for Birds." *European Journal of Wildlife Research*, 62. DOI: 10.1007/s10344-016-1025-x.
- [5] Lin, T., et al. (2017). "Focal Loss for Dense Object Detection." *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. DOI: 10.1109/ICCV.2017.324.
- [6] N. Mamat, M. F. Othman and F. Yakub, "Animal Intrusion Detection in Farming Area using YOLOv5 Approach," 2022 22nd International Conference on Control, Automation and Systems (ICCAS), Jeju, Korea, Republic of, 2022, pp. 1-5, doi: 10.23919/ICCAS55662.2022.10003780.
- [7] Noreen, Umber & Bounceur, AHCÈNE & Clavier, Laurent. (2017). A study of LoRa low power and wide area network technology. 1-6. 10.1109/AT-SIP.2017.8075570.

- [8] M. Saari, A. M. bin Baharudin, P. Sillberg, S. Hyrynsalmi and W. Yan, "LoRa — A survey of recent research trends," 2018 41st International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO), Opatija, Croatia, 2018, pp. 0872-0877, doi: 10.23919/MIPRO.2018.8400161.
- [9] S. Devalal and A. Karthikeyan, "LoRa Technology - An Overview," 2018 Second International Conference on Electronics, Communication and Aerospace Technology (ICECA), Coimbatore, India, 2018, pp. 284-290, doi: 10.1109/ICECA.2018.8474715.
- [10] B. Kellenberger, M. Volpi and D. Tuia, "Fast animal detection in UAV images using convolutional neural networks," 2017 IEEE International Geoscience and Remote Sensing Symposium (IGARSS), Fort Worth, TX, USA, 2017, pp. 866-869, doi: 10.1109/IGARSS.2017.8127090.
- [11] G. Chen, T. X. Han, Z. He, R. Kays, T. Forrester, Deep convolutional neural network based species recognition for wild animal monitoring, in: 2014 IEEE International Conference on Image Processing (ICIP), IEEE, 2014, pp. 858–862
- [12] A. Gomez, G. Diez, A. Salazar, A. Diaz, Animal identification in low quality camera-trap images using very deep convolutional neural networks and confidence thresholds, in: International Symposium on Visual Computing, Springer, 2016, pp. 747–756.
- [13] M. D. Zeiler, R. Fergus, Visualizing and understanding convolutional networks, in: Computer vision–ECCV 2014, Springer, 2014, pp. 818–833.
- [14] Shanaka Gunasekara, Maleen Jayasuriya, Nalin Harischandra, Lilantha Samaranyake, and Gamini Dissanayake, "A Convolutional Neural Network Based Early Warning System to Prevent Elephant-Train Collisions," 2021 IEEE 16th International Conference on Industrial and Information Systems (ICIIS), 2021, pp. 1-6, doi: 10.1109/ICIIS53135.2021.9660651
- [15] Gómez, A., Salazar, A., & Vargas, F. (2017). Towards automatic wild animal monitoring: Identification of animal species in camera-trap images

- using very deep convolutional neural networks. *Ecological Informatics*.
<https://doi.org/10.1016/j.ecoinf.2017.07.004>
- [16] R. M. Ghodhbane, A. Hernandez and S. Janfaoui, "A Pressure and Temperature Wireless Sensing Network Communicating with LoRa Modulation," 2024 IEEE Symposium on Computers and Communications (ISCC), Paris, France, 2024, pp. 1-3, doi: 10.1109/ISCC61673.2024.10733589.
 - [17] P. Shukla, I. Dua, B. Raman, and A. Mittal, "A computer vision framework for detecting and preventing human-elephant collisions," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV) Workshops*, Oct 2017.
 - [18] Y. Xue, T. Wang, and A. K. Skidmore, "Automatic Counting of Large Mammals from Very High Resolution Panchromatic Satellite Imagery," *Remote Sensing*, vol. 9, no. 9, p. 878, Aug. 2017. [Online]. Available: <http://www.mdpi.com/2072-4292/9/9/878>
 - [19] Sandler, Mark & Howard, Andrew & Zhu, Menglong & Zhmoginov, Andrey & Chen, Liang-Chieh. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. 4510-4520. 10.1109/CVPR.2018.00474
 - [20] J. G. Malayao, I. M. Bungay and A. N. Yumang, "Image Classification of Gamefowl Breeds Using MobileNet V2," 2024 8th International Conference on Electrical, Telecommunication and Computer Engineering (ELTICOM), Medan, Indonesia, 2024, pp. 171-175, doi: 10.1109/ELTICOM64085.2024.10864657.

APPENDIX

- Program used in the project

IMAGE PROCESSING MODULE CODE

```
import cv2
import tensorflow as tf
import numpy as np
import os
import serial

# Serial communication setup
ser = serial.Serial('COM3', 9600, timeout=1)

# Load labels
with open('labelmap.txt', 'r') as f:
    labels = [line.strip() for line in f.readlines()]
if labels[0] == '???':
    del labels[0]

# Load TensorFlow Lite model
interpreter = tf.lite.Interpreter(model_path="detect.tflite")
interpreter.allocate_tensors()
input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()
print("Waiting for IR sensor trigger...")
cap = None
try:
    exit_program = False
    buffer = ""
    while not exit_program:
        if ser.in_waiting > 0:
            char = ser.read().decode('utf-8', errors='ignore')
            buffer += char
            if char == '#':
                ir_data = buffer.strip('#')
                buffer = ""
                if ir_data == 'a':
                    print("Object detected. Starting image processing...")
```

```

if cap is None or not cap.isOpened():
cap = cv2.VideoCapture(0)
if not cap.isOpened():
print("Failed to open camera.")
break
while True:
if ser.in_waiting > 0:
char = ser.read().decode('utf-8', errors='ignore')
buffer += char
if char == '#':
ir_data = buffer.strip('#')
buffer = ""
ret, img_org = cap.read()
if not ret:
print("Failed to capture image.")
break
key = cv2.waitKey(1) & 0xFF
if key == 27:
break
img = cv2.cvtColor(img_org, cv2.COLOR_BGR2RGB)
img = cv2.resize(img, (300, 300))
img = img.reshape(1, 300, 300, 3).astype(np.uint8)
interpreter.set_tensor(input_details[0]['index'], img)
interpreter.invoke()
boxes = interpreter.get_tensor(output_details[0]['index'])
classes = interpreter.get_tensor(output_details[1]['index'])
scores = interpreter.get_tensor(output_details[2]['index'])
for i in range(boxes.shape[1]):
if scores[0, i] > 0.7:
object_name = labels[int(classes[0, i])]
if (object_name=="elephant" or object_name=="person" or object_name=="cow" or
object_name=="sheep" or object_name=="bear" or object_name=="dog" or

```

```

object_name=="zebra" or object_name=="giraffe" or object_name=="horse"):
box = boxes[0, i, :]
x0 = int(box[1] * img_org.shape[1])
y0 = int(box[0] * img_org.shape[0])
x1 = int(box[3] * img_org.shape[1])
y1 = int(box[2] * img_org.shape[0])
cv2.rectangle(img_org, (x0, y0), (x1, y1), (255, 0, 0), 2)
cv2.rectangle(img_org, (x0, y0), (x0 + 100, y0 - 30), (255, 0, 0), -1)
cv2.putText(img_org, object_name, (x0, y0), cv2.FONT_HERSHEY_SIMPLEX, 1,
(255, 255, 255), 2)
print(object_name)
ser.write(b'e')
else:
ser.write(b'n')
cv2.imshow('Image Processing', img_org)
cap.release()
cap = None
cv2.destroyAllWindows()
elif ir_data == 'b':
if cap is not None and cap.isOpened():
cap.release()
cap = None
cv2.destroyAllWindows()
key = cv2.waitKey(1) & 0xFF
if key == 27:
exit_program = True
finally:
if cap is not None and cap.isOpened():
cap.release()
ser.close()
cv2.destroyAllWindows()

```

TRANSMITTER CODE

```
#include <TinyGPS++.h>
#include<SoftwareSerial.h>
#include <SPI.h>
#include <LoRa.h>
SoftwareSerial ss(D3,D4); // RX, TX
TinyGPSPlus gps;
#define irsensor D1
#define NSS D8
#define RESET D0
#define DIO0 D2
String Phoneno[] = "+919497154283";
String phoneno;
float latitude,longitude;
int object;
void setup() {
  Serial.begin(9600);
  ss.begin(9600);
  ss.println("AT");
  delay(1000);
  ss.println("AT+CMGF=1");
  delay(1000);
  LoRa.setPins(NSS, RESET, DIO0);
  if (!LoRa.begin(433E6))
  {
    while (1);
  }
  pinMode(irsensor,INPUT);
}
void loop() {
  object=digitalRead(irsensor);
```

```

if(!object)
{
Serial.print("a#");
}
else
{
Serial.print("b#");
}
if (ss.available() > 0)
{
for(int i=0;i<500;i++)
{
if (gps.encode(ss.read()))
{
gpslocation();
}
}
}
if(Serial.available()>0) {
char X=Serial.read();
if(X=='e')
{
String message = "warning";
LoRa.beginPacket();
LoRa.print(message);
LoRa.endPacket();
for(int i=0;i<=3;i++)
{
sendGPSThroughSMS(Phoneno[i],latitude,longitude);
}
}
}
}

```

```

delay(100);
}
void gpslocation()
{
//Serial.println("from gps fun");
if(gps.location.isValid())
{
latitude = (gps.location.lat());
longitude = (gps.location.lng());
Serial.print("LAT: ");
Serial.println(latitude, 6); // float to x decimal places
Serial.print("LONG: ");
Serial.println(longitude, 6);
return;
}
}

void sendGPSThroughSMS(String phoneno,float latitude, float longitude) {
String message = "Latitude: " + String(latitude, 6) + "Longitude: " + String(longitude,
6);
// Sending SMS
ss.print("AT+CMGF=1\r");
delay(1000);
ss.print("AT+CMGS=\r");
ss.print(phoneno);
ss.println("\r");
delay(1000);
ss.println("Alert"+message);
ss.print((char)26);
}
}

```

RECEIVER CODE

```
#include <LiquidCrystal_I2C.h>

#include <SPI.h>
#include <LoRa.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);

#define NSS 10
#define RESET 9
#define DIO0 8

String com;

void setup() {
  Serial.begin(9600);
  lcd.init();
  lcd.backlight();
  LoRa.setPins(NSS, RESET, DIO0);
  pinMode(3,OUTPUT);
  if (!LoRa.begin(433E6))
  {
    Serial.println("LoRa initialization failed!");
    lcd.clear();
    lcd.print("LoRa Init Failed!");
    while (1);
  }
}

void loop() {
  int packetSize = LoRa.parsePacket();
  if (packetSize)
  {
    String receivedData = "";
    while (LoRa.available())
    {
      receivedData += (char)LoRa.read();
    }
  }
}
```

```

}
Serial.println("Received Data: " + receivedData);
if (receivedData == "warning") {
  Serial.println("elephant");
  lcd.setCursor(5,0);
  lcd.print("obstacle");
  lcd.setCursor(3,1);
  lcd.print("detected");
  digitalWrite(3,HIGH);
  delay(10000);
  digitalWrite(3,LOW);
  lcd.clear();
}
else {
  Serial.println("else");
  lcd.clear();
}
}
}
}

```